



MACQUARIE
University
SYDNEY • AUSTRALIA

COMP3850 Project Deliverable Certificate

Name of Deliverable	<i>D2- Feasibility, Team Manual, Project Plan and Requirements/Scoping Document</i>
Date Submitted	<i>04 / 09 / 2025</i>
Project Group Number	<i>22</i>
Rubric stream being followed for this deliverable (highlight one) <i>Note: the feasibility study has the same rubric for all streams.</i>	SOFTWARE Rubric GAMES Rubric CYBERSECURITY Rubric DATA SCIENCE Rubric

We, the undersigned members of the above Project Group, collectively and individually certify that the above Project Deliverable, as submitted, **is entirely our own work**, other than where explicitly indicated in the deliverable documentation.

INITIALS	SURNAME	GIVEN NAME	STUDENT NUMBER	SIGNATURE (IN-PERSON OR DIGITAL)
<i>KAA</i>	<i>Ashraf</i>	<i>Khawaja Abeer</i>	<i>47650249</i>	<i>Abeer</i>
<i>EP</i>	<i>Chairudin</i>	<i>Edward Pandiya</i>	<i>47977523</i>	<i>Edward</i>
<i>EK</i>	<i>Kok</i>	<i>Eric</i>	<i>46108572</i>	<i>eric</i>
<i>YD</i>	<i>Diab</i>	<i>Yousef</i>	<i>47359099</i>	<i>Yousef</i>
<i>AA</i>	<i>Abdelghaffar</i>	<i>Amr</i>	<i>47872411</i>	<i>Amr</i>
<i>AG</i>	<i>Gupta</i>	<i>Anirudh</i>	<i>47547081</i>	<i>Anirudh</i>

NB: please write all details clearly (if handwritten).
© Macquarie University, 2021

List of tasks completed for the deliverable and activities since last deliverable certificate with totals for each individual team member and whole team

(copy individual total row for each member and copy pages if more pages needed)

Performed by <i>(Student Names)</i>	Dur atio n <i>(hrs)</i>	Co mpl exit y <i>(L, M, H)</i>	Name of task	Checked by <i>(Initials)</i>
Khawaja	2	H	Wrote Feasibility Report Section 4.4, 4.5, 4.6 alternatives as suggested by convenor	AG
Khawaja	1	M	Wrote Feasibility Report Section references, assumptions, resources as convenor said in feedback	AG
Khawaja	0.5	M	Contributed to Team Manual in my section 4 after convenor's feedback	AG
Khawaja	1	M	Incorporated feedback from sponsor & convenor into reports	AG
Khawaja	2	M	Drafted SRS points across all sections including (Purpose, Interfaces, Functional)	AG
Khawaja	1.5	H	Wrote SRS Section 4 (External Interface Requirements) in detailed report style	AG
Khawaja	2	H	Drafted Project Plan points (Risk Management, Resource Management, Change Management, Standards)	AG
Khawaja	1.5	M	Wrote Project Plan section 2 Risk Management	AG
Khawaja	2.5	M	Created Scrum and Kanban boards in Jira for project management	AG
Khawaja	1	M	Meeting with Sponsor - asked technical & NLP ambiguity questions	AG
Khawaja	1	M	Drafted weekly reflections for team deliverables	AG
Khawaja	1.5	H	Researched on models to input weights manually and LDA transformers as discussed with sponsor	AG
Khawaja	0.5	H	Made context diagram and rechecked sequence diagram	AG
Khawaja	1.5	M	Prepared DFD pipeline and Risk Matrix diagrams	AG
Khawaja	1.5	M	Wrote Project Progress report (Research & Training, Tasks, V&V)	AG
Khawaja	1	L	Wrote individual weekly reflections (summarising tasks done and Jira setup)	AG
Khawaja	0.5	L	Reviewed and proofread team deliverables before submission	AG
Eric	2	M	Contributed in drafting ideas for SRS and project plan documents.	AG
Eric	2.5	M	Completed and refined section 2.1, 2.2, 2.3 and 3.1 of SRS document.	AG
Eric	2.5	M	Completed and refined section 3 of project plan document.	AG

Eric	2.5	M	Created diagrams to illustrate product perspective and data flow (Sequence diagram and data flow diagram).	AG
Eric	0.5	L	Wrote and submitted a weekly report for week 4 for the team.	AG
Eric	1	L	Documented meeting minutes for sponsor meetings and team meetings	AG
Eric	2	M	Wired the API routes to establish a working pipeline.	AG
Eric	2	M	Serialised the model fields for query and post object.	AG
Eric	3	H	Researched about public APIs and data fetching from multiple data sources.	AG
Edward	1	M	Making bullet point for project progress and SRS	AG
Edward	1	M	making serializer like a translator from django to JSON, and the other way around	AG
Edward	2.5	M	Setting up the database and migrate it, inside the django program, including the post and pre search + username	AG
Edward	2.5	M	Doing section 5.3, 5.4 and 6 in SRS	AG
Edward	3	M	doing the state of purpose, Scope , and assumption inside the project plan	AG
Edward	1	L	Formatting the feedback from sponsor related to project plan, and SRS document	AG
Edward	3	M	Research about data fetch on django and how to do it	AG
Edward	3	M	doing data fetch on my code, still progressing haven't done and push to github yet	AG
Anirudh	2	M	Rewrote section 1 of the Feasibility report	YD
Anirudh	3	M	Drafted points across the SRS Document	YD
Anirudh	3	M	Drafted points across the Project Plan Document	YD
Anirudh	2	M	Wrote section 5 (Quality Management) of the Project Plan in report style.	YD
Anirudh	2	M	Wrote section 1 of the SRS in report style	YD
Anirudh	4	M	Created User stories and Acceptance criteria for the platform for the ease of team	YD
Anirudh	1.5	L	Formatted sponsor reviews for the documents to be submitted in Deliverable 2	YD
Anirudh	0.5	L	Prepared a weekly individual reflection	YD
Anirudh	2	M	Researched Hugging Face and other similar platforms that provide pre-trained models	YD
Yousef	2	M	Contributed to the SRS draft and Project Plan draft	KAA
Yousef	3	M	Refined all of section 5 for the Feasibility Report and section 3 in the Team Manual, based on feedback	KAA
Yousef	2	M	Completed 5.1, 5.2 for the SRS .	KAA
Yousef	3	M	Completed section 6 and 8 for the Project Plan	KAA
Yousef	1	L	written weekly individual report reflections	KAA
Yousef	1	L	Submitted the weekly report for week 5	KAA
Yousef	4	H	Designed screen designs for further feedback	KAA
Yousef	3	H	Further research on CSS tailwind	KAA

Yousef	2	H	Explored more on UI design	KAA
Amr Abdelghaffar	1	L	Improving Section 2 of the Feasibility Report and 5.2 of the Team Manual to address the feedback given.	YD
Amr Abdelghaffar	1	M	Working on first draft of Project Plan	YD
Amr Abdelghaffar	2	M	Researching scoring techniques to be used in the application, such as lexical, semantic, and engagement.	YD
Amr Abdelghaffar	1	M	Trying NLP techniques, such as lexical and semantic scoring, on sample data	YD
Amr Abdelghaffar	0.5	L	Working on first draft of SRS	YD
Amr Abdelghaffar	1.5	M	Creating full draft of Section 4 of Project Plan	YD
Amr Abdelghaffar	1.5	M	Finalising Section 4 of Project Plan	YD
Amr Abdelghaffar	1	M	Creating full draft of assigned parts of Section 2 of SRS	YD
Amr Abdelghaffar	1	M	Finalising assigned parts of Section 2 of SRS	YD
Amr Abdelghaffar	1	L	Adjusting Section 4 of the Project Plan, and Section 2 of the SRS, based on sponsor feedback.	YD
Amr Abdelghaffar	2	M	Proofreading and providing some fixes to the finalised Project Plan and SRS documents	YD
<i>Yousef Diab Total</i>	21			
<i>Khawaja Abeer Total</i>	22.5			
<i>Anirudh Gupta Total</i>	20			
<i>Eric Kok Total</i>	18			
<i>Edward Pandiya Chairudin Total</i>	17			
<i>Amr Abdelghaffar</i>	13.5			
Team Total				

Statement for Marker

Clarify if there are any specific matters you want to clarify to the marker. For example, if you have convenor approval to use a rubric merged from more than one stream (If so, please provide the approved rubric here). Leave blank if there are no special instructions.



Robotic Marketer Pty Ltd

Feasibility Study

< AI-Powered Web Application for Bot Listening >

File location: H:\Change\Feasibility_Study.doc

Version <01>, <11/08/2025>

Contents

1.	Introduction	3
1.1.	Problem Identification.....	3
1.2.	Opportunities.....	4
1.3.	Mandates.....	5
1.4.	Assumptions.....	5
2.	Current Situation	5
3.	Benefits	6
3.1.	Tangible Benefits.....	6
3.2.	Intangible Benefits.....	7
4.	Alternative Solutions	7
4.1.	Alternative 1.....	7
4.1.1.	Advantages.....	7
4.1.2.	Disadvantages.....	8
4.2.	Alternative 2.....	8
4.2.1.	Advantages.....	8
4.2.2.	Disadvantages.....	9
4.3.	Alternative 3.....	9
4.3.1.	Advantages.....	9

4.3.2.	Disadvantages.....	10
4.4.	Alternative 4.....	10
4.4.1.	Advantages.....	10
4.4.2.	Disadvantages.....	10
4.5.	Alternative 5.....	11
4.5.1.	Advantages.....	11
4.5.2.	Disadvantages.....	11
4.6.	Alternative 6.....	11
4.6.1.	Advantages.....	11
4.6.2.	Disadvantages.....	12
5.	Recommended Solution	12
5.1	Structured Comparison.....	12
5.2	Decision.....	13
5.3	Key Reasons Why.....	13
5.4	Implementation Road Map Plan.....	14
5.5	Key Success Metrics.....	15
5.6	Risk and Mitigations.....	15

1. Introduction

This report is a feasibility study for Robotic Marketer Pty Ltd to assess the potential development of an AI-powered research application. It provides a structured analysis of the project's scope, including the problem context, opportunities, mandates, and assumptions guiding development. The report also outlines the current situation, evaluates tangible and intangible benefits, and considers alternative solutions before recommending the most viable option. The document is intended for Robotic Marketer stakeholders as well as potential investors to support informed decision-making about project initiation.

1.1. Problem Identification

On average, a knowledge-gathering worker spends more than two hours a day gathering information, according to [1]. The process is highly repetitive, inconsistent, and time-consuming. Without AI-driven relevance scoring, organizations face productivity loss, delayed decisions and limited ability to scale research operations. After implementing the project proposed by Robotic Marketer Pty Ltd, they aim to address the following issues:

- Manual searching and ranking are time-consuming tasks. After automating this process, the application can be left to the job on its own. The hours thus saved can be employed somewhere else.
- Having a load of resources can make it harder for research gathering workers to rank the articles based on their relevance. The process involved checking for the credibility of the said article and then accurately ranking them based on their relevance. The platform Robotic Marketer proposes to automate this process entirely, increasing efficiency in research gathering and leaving research gathering workers with more hours on their hand for more important stuff.
- Current manual methods fail to keep up with the increasing demand of the world. Without automation, the process will keep failing to do so and only increase the chances of inaccuracies with manual labour

While addressing these business issues, the development of the application also introduces its own set of challenges:

- **Data Access:** The project requires us to rely on third-party APIs, which may have rate limits, costs or licensing restrictions.
- **Data Quality:** Gathering articles from different resources brings its own set of challenges. Ensuring consistency, credibility and accuracy are concerns that need to be addressed while preprocessing.
- **Legal and compliance risks:** The team will need to navigate intellectual property rights and privacy regulations when pulling data from external resources.

- **Model Performance:** The project requires a ranking model that balances accuracy, speed and computational costs.
- **Scalability:** Building a system architecture that can handle growth while maintaining the security of API keys and user data.

1.2. Opportunities

The proposed application creates several opportunities for Robotic Marketer Pty Ltd and its stakeholders. These can be grouped into four categories:

- **Market Growth:** Access to a new audience that was previously out of Robotic Marketer Pty Ltd reach. This includes, but is not limited to, research analysts, students and research-based organizations. Such market growth also enables the Robtic Marketer to build a strong knowledge base, which may help the organization to position itself as a leader in the field of research automation.
- **Financial Opportunities:** The firm can generate early subscription revenue by releasing an MVP within 3-4 months. Other revenue streams would include revenue through targeted in-app advertising or providing custom enterprise solutions. The problems solved by the product almost guarantee an ROI within a short period of time through time savings and increased productivity throughout the organization.
- **Competitive Edge:** With the proposed product, Robotic Marketer establishes itself as a provider of cutting-edge, AI-powered tech in the market. They also distinguish themselves from already established research platforms by introducing AI-powered ranking system for articles.
- **Scalability:** Once the platform has been used for some time, the knowledge base established can be used to detect early trends. Research articles can be pre-ranked based on the trend and it further reduces the time for searching and ranking. The application has the potential to scale globally fairly easily.

The application also provides its own set of opportunities:

- **Innovation:** The project requires the team to develop a new model and implement new strategies. The model thus developed can be reused for future projects.
- **Data infrastructure:** Build scalable data pipelines with MongoDB and cloud computing platforms, creating a foundation for future AI applications.
- **Ethical AI leadership:** Showcase Robotic Marketer's commitment to responsible AI by reducing dataset bias and aligning with ethical standards.

1.3. Mandates

The project must meet several statutory and organisational requirements. It must comply with privacy regulation and relevant government laws, including the Australian Privacy Act 1988, as well as international data protection standards such as GDPR and CCPA to ensure lawful data storage and processing. The AI models must adhere to ethical guidelines. Measures should be taken to remove bias from the training dataset. Any Machine language model or datasets used should have appropriate licensing for commercial use to avoid violation of intellectual property rights.

Other mandates include adherence to API usage policies from source platforms, implementation of front end security measures such as securing API keys and sanitizing untrusted input. Measures to include user tracking to track their activities to track unusual activities. The system must also meet WCAG 2.1 AA accessibility standard to ensure inclusive usability across all audiences.

1.4. Assumptions

The most important assumption for the success of this project is the availability of public API and datasets for data retrieval. The project also requires access to high quality data for model testing and tuning. It is also assumed that cloud based computing resources, such as AWS and Azure, will be available for AI model development. Mongo DB will be made accessible to store huge datasets to reproduce results and other purposes.

From a technical standpoint, Django will be the backend framework with access to basic NLP libraries. Branding guidelines will be provided early on to ensure a consistent user interface and experience. The target browsers will include latest versions of Chrome, Edge and Safari on both desktop and mobile and the system will temporarily cache responses where appropriate.

From a risk perspective, it is assumed that API access will remain available at current rate limits and pricing and that no major policy change will occur that would impact data retrieval.

2. Current Situation

There is a clear gap in the market for a tool capable of aggregating and ranking individuals and organizations by topic relevance across platforms. The creation of such a tool would enable Robotic Marketer Pty Ltd to provide their clients with highly relevant information on current conversations regarding their specific industry and location. Existing tools only provide broad social listening, lacking the precision needed to obtain the relevant information from a specific niche industry.

Additionally, no system exists to automate semantic similarity matching between user queries and online conversations, reducing the precision and scalability of the company's operations. The current reliance on more manual methods is highly inefficient, due to the time-consuming nature, leading to the increased difficulty to scale. The inefficiency also has the potential to reduce the accuracy and relevancy of the results, as it means less data can be analysed in the same time frame.

Furthermore, the lack of specialised tools that tailor output to both industry and location makes it challenging for Robotic Marketer's clients to obtain the needed data to create an appropriate marketing plan. Due to the lack of highly relevant information, the clients risk basing their marketing strategy on incomplete or irrelevant information. This leads to a competitive disadvantage for the clients, as it can mean not being able to capitalise on emerging trends in their niche in a timely manner.

3. Benefits

The proposed AI-driven lead discovery and market intelligence platform will provide both tangible and intangible benefits, enabling measurable cost and time savings while also delivering strategic advantages in client relationships, brand positioning, and operational efficiency.

3.1. Tangible Benefits

Cost Savings

- Reduce labour costs by cutting manual research hours by up to **60%**.
- Lower marketing expenditure by **20–30%** through higher-precision targeting and reduced ad spend wastage.
- Increase revenue by **15–20%** via improved conversion rates from higher-quality leads.

Time Savings

- Reclaim **3–5 staff hours per week** per researcher by automating lead discovery.
- Accelerate decision-making cycles by **40%** through instant AI-driven insight generation.
- Increase lead generation throughput by **50%**, enabling faster campaign rollouts.

New Products and Services

- Launch premium AI Discovery Bot features with potential **15% uplift** in upsell revenue.
- Offer personalised AI-driven intelligence reports as an added-value service to top-tier clients.
- Enhance lead targeting accuracy via semantic relevance scoring, directly benefiting marketing, sales, and research teams.

3.2. Intangible Benefits

- **Increased client trust** through transparent, data-backed AI insights, improving credibility and authority.
- **Brand differentiation** as a modern, intelligence-led discovery platform, positioning the organisation ahead of competitors.
- **Improved inter-departmental collaboration** through consistent, standardised data formats across teams.
- **Higher employee morale** by removing repetitive, low-value manual work, allowing staff to focus on strategic and creative tasks.
- **Strengthened client confidence** in the organisation's ability to deliver accurate, timely, and relevant market insights.

4. Alternative Solutions

4.1. Alternative 1 – In-House AI-Powered Web Application

Developing an AI-powered web application entirely in-house would give Robotic Marketer complete ownership over the design, functionality and underlying technology. This approach would allow the team to craft a solution specifically tailored to client requirements while ensuring seamless integration with the company's existing AI platform. By keeping development internal, the business retains full control over data usage, privacy protocols and system architecture, which can be crucial for maintaining competitive advantage.

4.1.1. Advantages

- **Scalability and adaptability:** Can evolve with client needs and market trends.
Example: Canva's in-house branding kit allowed them to respond rapidly to enterprise feature requests without third-party delays.
- **Full control over design, functionality, and data usage:** Enables unique features and ensures compliance with privacy/security standards.
- **Seamless integration with Robotic Marketer's existing AI platform:** Allows for a unified user experience and efficient data flow.
- **Future innovation potential:** Could integrate with voice assistants or chat-based research tools for hands-free, conversational insights (similar to Bloomberg Terminal's voice search).
- **Continuous improvement through machine learning feedback loops:** AI models can refine outputs over time based on user interactions, much like Netflix's recommendation system.

4.1.2. Disadvantages

- **High upfront and ongoing costs:** Developing an in-house AI-powered platform would require a substantial initial investment in both time and resources, as well as ongoing expenses for infrastructure, updates and maintenance. The overall financial commitment could be significant compared to a third party solution.
- **Potential resource strain:** May divert focus from other high-priority product features in the pipeline.
- **Longer testing cycles:** AI features often require iterative testing, model retraining and tuning, which can extend quality assurance (QA) timelines compared to non-AI systems.
- **Ongoing dependency on third-party APIs:** Even with an in-house build, real-time data sources may still rely on external providers whose stability is outside company control.

4.2 Alternative 2 - No Action

4.2.1. Advantages

A no action decision would imply that the organization would not have any initial financial expenditure in terms of the development or acquisition of an AI-powered web application. It could be especially advantageous in case it is necessary to address the issue of underfunding because all the funds available could be spent on other business needs. The company also evades risks that could arise as a result of AI implementation including system and security breaches and integration failures by not developing new technology. Also, there are no requirements of technical support on a continuous basis, software updates, as well as specific maintenance teams to take care of since it comes without a new AI-powered system to manage. This will enable the company to wholly focus on streamlining and enhancing the available features of its AI marketing strategy so that current equipment and procedures can perform to their best capacity without being distracted by other novel untested projects.

Bulletpoint version:

- No financial investment
- Avoids technology risks
- No maintenance burden
- Allows focus on existing AI marketing features

4.2.2. Disadvantages

Although the no action option is less expensive and less risky, it has major disadvantages. Without AI driven web applications, data will still be collected manually which is commonly slower, resource demanding and subject to errors involving the idle hand. This may hamper the speed with which the organization can determine the trends, consumer habits, or market changes. Also, the company might lose to the extent of growth that competitors can achieve because they pursue AI solutions and integration that enables them to automate, use predictive analytics, and become more engaging to their customer base. This may eventually result in a competitive disadvantage in the long run since competing organizations using AI and

machine learning will have quicker access to insights, perform more efficiently, and be better equipped to act on the needs of the market. After all, failure to do anything might lead to the firm being left behind in a world that is becoming more and more technology oriented.

Bullerpoint version:

- Continued manual data collection
- Missed growth opportunities
- Falling behind competitors adopting AI-powered tools

4.3. Alternative 3 – White-Label AI Integration

In this option, the development team would integrate and customise a commercially available white-label AI lead generation and market intelligence platform. These platforms typically come pre-built with core capabilities such as data aggregation, trend detection, predictive analytics, and dashboard visualisation, while still allowing for branding, workflow tailoring, and API extensions to suit specific organisational needs.

4.3.1. Advantages

- **Faster Deployment:** Potentially functional within **8–12 weeks**, significantly faster than a full in-house build.
- **Lower Upfront Costs:** Subscription/licensing model reduces initial capital expenditure.
- **Scalability:** Flexible resource scaling as demand grows.
- **Mature Core Features:** Access to ready-made, tested functionalities (data aggregation, analytics, visualisation).
- **Customisation Potential:** Ability to add branding, adjust workflows, and integrate with other systems via APIs.
- **Regulatory Compliance:** Vendor solutions can be selected based on adherence to GDPR, CCPA, and commercial-use licensing.

4.3.2. Disadvantages

- **Ongoing Costs:** Recurring subscription or licensing fees may be substantial over time.
- **Vendor Dependence:** Reliance on a third-party provider for core infrastructure and updates.
- **Customisation Limits:** May not allow deep, highly tailored changes to underlying architecture.
- **Data Security Risks:** Sensitive data handling depends on vendor's security measures.

- **Feature Constraints:** Must operate within the vendor's feature set unless they offer extended API capabilities.

4.4 Alternative 4 – Hybrid Approach (In-House + Vendor Partnerships)

This approach combines building core proprietary components in-house while leveraging vendor solutions for non-critical or infrastructure-heavy aspects (e.g., cloud storage, data pipelines, or summarisation APIs).

4.4.1. Advantages

- **Balanced cost and control:** Internal team retains ownership of critical IP, while vendor solutions reduce initial costs and speed up development.
- **Faster MVP delivery:** Vendor APIs and tools accelerate initial deployment while core AI features are built in-house.
- **Reduced risk:** Vendor dependency is limited to non-core features, while sensitive data and core models remain internally managed.
- **Scalability and flexibility:** Allows modular replacement of vendor tools with in-house systems over time.

4.4.2. Disadvantages

- **Integration complexity:** Combining in-house systems with third-party solutions may create technical dependencies.
- **Dual costs:** Requires managing both vendor subscriptions and in-house development expenses.
- **Long-term transition burden:** Vendor components may eventually need to be replaced, increasing future workload.

4.5 Alternative 6 – Open-Source First Build

Assemble the platform primarily from open-source components (e.g., Django REST, OpenSearch/Elasticsearch, sentence-transformers/SBERT, Haystack) with minimal or no proprietary licenses.

4.5.1. Advantages

- **Low licensing cost:** Minimises recurring fees; favourable for academic/innovation settings.
- **Transparency and extensibility:** Full access to source for audits, security hardening, and custom features.

- **Community support:** Large ecosystems, plugins, and tutorials shorten development.
- **Avoids vendor lock-in:** Components can be swapped out with limited contractual friction.

4.5.2. Disadvantages

- **Self-managed burden:** Patching, upgrades, security scans, and uptime are the team's responsibility.
- **Integration complexity:** Getting components to interoperate reliably requires solid DevOps discipline.
- **No warranties:** Community software comes without formal support unless a support contract is added.
- **Performance tuning required:** Achieving production-grade latency/throughput may need specialist skills.

4.6 Alternative 5 – Outsourced Custom

Development (Specialist Vendor)

Engage a specialist software vendor to design and build a bespoke solution under a services contract, with Robotic Marketer owning the IP per agreement.

4.6.1. Advantages

- **Speed and expertise:** Access to an experienced delivery team with proven data/NLP accelerators reduces ramp-up time.
- **Predictable delivery:** Fixed-bid or milestone-based contracts provide clearer cost/schedule control.
- **Knowledge transfer:** Vendor handover and shadowing can upskill internal staff for post-launch ownership.
- **SLA-backed quality:** Commercial SLAs for defects, security, and response times reduce operational risk.

4.6.2. Disadvantages

- **Higher TCO:** Premium day rates; change requests can escalate cost.
- **Control and lock-in risk:** Dependency on vendor processes/tooling; IP must be contractually secured.
- **Alignment overhead:** Extra effort to align requirements, review artefacts, and manage delivery governance.
- **Less organic capability growth:** Internal team gains fewer hands-on build cycles during the initial phase.

5. Recommended Solution

5.1 Structured Comparison

Criteria	Alt 1: In-House Build	Alt 2: No Action	Alt 3: White-La bel Integratio n	Alt 4: Hybrid (In-Ho use + Vendor)	Alt 5: Outsourc ed Custom Dev	Alt 6: Open-Sou rce First Build
Up front cost	High (team/infrastru cture)	None	Low–Med ium (license/se tup)	Mediu m (mixed spend)	High (vendor fees)	Low (licenses) / Medium (engineerin g)
Deployment speed	Medium (MVP ~3–4 mo)	Immedi ate	Fast (8–12 wks)	Fast for non-cor e pieces	Medium (contract setup)	Medium (integratio n effort)
Control over data/IP	Full	No change	Limited (vendor core)	High on core; vendor for non-cor e	Medium (contractu al)	High (code owned)
Customization/flex ibility	High	None	Medium (within vendor limits)	High on core	High (via contract)	High (source available)
Integration fit	Seamless with current platform	None	Medium (API-depe ndent)	High (design ed that way)	Medium (align via SOW)	Medium– High (needs glue code)
Compliance (GDPR/CCPA/Pri vacy Act, WCAG 2.1 AA)	Direct control to meet mandates	Manual process es remain	Vendor-de pendent	Control led on core, vendor	Contractu al SLAs	Team-man aged; transparent

				attestation on rest		
Ongoing cost	Moderate (maintenance)	None	High (recurring fees)	Medium (mixed)	High (support/CRs)	Low–Medium (ops time)
Key risks	Team capacity, time	Strategic stagnation	Lock-in, data/security	Integration complexity	Cost growth, vendor reliance	Integration/ops burden
Long term advantage	Strong differentiation	Weak	Parity, not unique	Strong on core, faster start	Medium (depends on vendor)	Strong (no lock-in)

5.2 Decision

Creating the AI web module inside the company may be the best route. It would sit right inside Robotic Marketer's platform and match the brand. For non-core items like infrastructure helpers, the team could still pick cheap open-source tools or small vendor utilities. The main ranking engine, data handling and user interface would stay under direct control.

5.3 Key Reasons Why

An internal build appears to give full ownership of data, intellectual property, security, and compliance (like GDPR, CCPA, WCAG 2.1 AA). It should also blend smoothly with the existing design system. Continuous improvements seem possible without waiting on a third-party roadmap. The trade-off is a larger upfront effort compared with a white-label solution. Yet the long-term cost, avoiding license fees and lock-in, may prove superior and fits the “no paid LLMs” rule. for us.

5.4 Implementation Road Map Plan

Phase 1 — Discovery & Deliverable 1 (Weeks 1–3)

Goal: try to line up scope, success measures, and the “no paid LLMs” rule.
 What we do: assign roles, set up quick chats, write a Feasibility draft plus a Team Handbook, sketch a simple SRS, pull together a rough Search-History view, note three filters – Subject, Location, Industry – and basic UI states such as loading, empty and error.
 Outputs: D1 packet, first-version wireframes, an early SRS outline.

Phase 2 — Tech Baseline & Docs Pack (Weeks 4–6)

Idea: prove the stack works and write down what we built.

Steps: spin up a Python 3.12 (maybe Django 5.2) app, hook it to MongoDB 8 via PyMongo, expose a light /api endpoint and a health check, tidy the wireframes, flesh out SRS sections 2.4, 2.5, 4.3 (stack & interfaces) and 4.1 (UI basics – keyboard path, skip link, visible focus).

Outputs: a dev server that actually runs, a verified DB link, a quick route list, and D2 – the Project Plan, full SRS, Team Handbook and Feasibility note.

Phase 3 — MVP & Prototype (Weeks 7–9)

Aim: get something that runs from end to end.

Tasks include: add a baseline ranking (lexical plus a tiny semantic hint), create a server-rendered Search and History with pagination limited to ten items per page, wire the three filters, keep response time around five seconds (no more than ten) for the first useful hit, connect input → ranking → output, store recent searches, make slow or down sources fail-soft so they don't hold everything up, drop in keyboard-only shortcuts (skip link, logical tab order, visible focus) and pass a quick Chrome/Edge/Safari smoke test, write a demo script draft.

Outputs: a working MVP that can be demoed, screenshot collection and brief test notes.

Phase 4 — Hardening & Docs/Training (Weeks 10–11)

Focus: tighten quality, improve accessibility, set up operations.

We fix bugs, clean up copy, boost speed (reuse cached results then refresh in background), run a basic WCAG 2.1 AA check, grow test coverage with unit and simple end-to-end cases, draft an operations run-book covering start/stop, env-vars and backups, and put together user and training notes.

Outputs: test summary, updated documentation, run-book and training handout.

Phase 5 — Demo & Handover (Week 12+)

Purpose: show the work, package it, hand it over.

We polish the demo and slide deck, rehearse, take feedback, bundle code, docs, run-book, config templates and a credential checklist, ask the sponsor for sign-off and list any stretch goals.

Outputs: a finished demo, a complete hand-off package and a recorded sign-off.

5.5 Key Success Metrics

Success may show as fast answers, clear gains in efficiency and safe work in sponsor's place. The system should give the first ranked result about 4 seconds on average. Result pages keep ten items so clicks stay quick. Relevance should lift work quality; at least thirty percent of top ten items need be on-topic during the pilot. The tool aims to cut manual research time by roughly fifty to sixty percent versus current baseline. At same time it must stay reliable and meet compliance: no blocking defects at demo, cross-browser checks pass Chrome Edge Safari, basic WCAG 2.1 AA items met, keyboard path, skip link, visible focus, clear headings. Privacy terms stay respected always: read-only mode, minimal logs, no client secrets. Adoption shown by five internal pilot users and two sponsor backed cases by beta end with positive review notes.

5.6 Risk and Mitigations

Biggest risks appear as external API volatility, rate limits, scope creep and limited student capacity. To soften API risk, the system will cache recent results, use polite back-off retries and fall back to partial data; a small curated fallback set remains for demos. Scope creep is limited by locking MVP in Phase 1, moving new ideas to a stretch list and holding short weekly check-ins with sponsor to confirm priorities. Capacity risk lowered by one-week sprints, small testable tasks and early integration demos that reveal issues early. Compliance and security handled with read-only integrations, server-side secrets, HTTPS with minimal privacy safe logs. Usability and accessibility is kept by keyboard only walkthroughs, cross-browser smoke tests and quick UI fixes during hardening so the prototype stays usable even when sources are slow or unavailable.

REFERENCES:

- IEEE Std 830-1998. *Recommended Practice for Software Requirements Specifications*. Institute of Electrical and Electronics Engineers.
 - W3C (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. <https://www.w3.org/TR/WCAG21/>
 - European Union. *General Data Protection Regulation (GDPR)*. <https://gdpr-info.eu/>
 - Australian Government. *Privacy Act 1988 (Cth)*.
 - Probst, S. (2019). *Reality Check: Still Spending More Time Gathering Instead of Analyzing*. Forbes Technology Council.
-

Abbreviations

- **AI** – Artificial Intelligence
 - **API** – Application Programming Interface
 - **GDPR** – General Data Protection Regulation
 - **MVP** – Minimum Viable Product
 - **NLP** – Natural Language Processing
 - **PII** – Personally Identifiable Information
 - **UI/UX** – User Interface / User Experience
 - **WCAG** – Web Content Accessibility Guidelines
-

Assumptions

Technology

- Public APIs and open datasets will remain accessible at current rate limits, costs, and terms of service throughout the project.
- Cloud services (MongoDB Atlas, Fly.io, AWS/Azure) will be stable and available for database, hosting, and computation.
- NLP/ML libraries such as Hugging Face, Scikit-learn, and PyTorch will remain free and open source under compatible licenses.
- The system will operate effectively on standard web browsers (latest versions of Chrome, Safari, Firefox, Edge).
- Project infrastructure (e.g., GitHub for version control, Jira for task management) will be reliable and accessible to all members.

Communication / Team / Sponsor

- Weekly team meetings (Friday 3 PM) will continue as scheduled, with additional ad hoc check-ins as needed.
- The sponsor (Robotic Marketer Pty Ltd) will provide feedback within 3–5 working days on deliverables, ensuring timelines are not disrupted.
- Communication tools (WhatsApp, Discord, Teams) will remain available for real-time collaboration.
- The convenor will be accessible for conflict resolution or guidance when required.
- All team members will remain committed to open communication, timely updates, and resolving conflicts constructively.

Resources

- Each team member will contribute an equitable workload, with flexibility during exam/assessment periods.
- Branding guidelines and design assets will be provided by the sponsor in a timely manner.
- Adequate computing resources (laptops with required specs, stable internet) are available to all team members.
- Training material (online tutorials, documentation, sponsor-provided resources) will be sufficient to address knowledge gaps in AI/NLP methods.
- No major cost or licensing barriers will arise beyond planned cloud and API usage.



<AI-Powered Web Application for Bot Listening>

Version <01>, <03/09/2025>

Contents

1. Statement of Purpose	2
2. Risk Management	2
2.1. ACS Code of Ethics Overview	2
2.2. Team Values	2
3. Resource Management	2
3.1. People	2
3.2. Hardware, Software, Other	2
4. Change management	
4.1. Managing requirements and scope change	
4.2 Version Control	
5. Quality Management	3
5.1. Managing Change	3
5.2. Conflict Resolution	3
6. Project Schedule	
7. Assumptions	
8. Standards/Templates/Appendices/Forms	

1. Statement of purpose [for plan] and scope [of project]

This project is aimed at designing and developing an AI-driven web-based application that can extract research articles on a large variety of online resources and use state-of-the-art machine learning (ML) and natural language processing (NLP) strategies to assign to each article a relevancy score. In this way, the system is designed to help solve one of the biggest problems of any researcher: the informational overload, the necessity to spend much time on finding the most useful and authoritative content. This application should save considerably on the time and other efforts used in manual searches so that users can concentrate more on analysis, interpretation and deriving insights. The major intended audience of this system will be research analysts, universities, students, data scientists and research-based organizations all who depend on up to date and quality information to carry out their duties.

Besides producing an effective research tool, the project will record the methodologies, tools and technologies used in developing the system and in this way, ensure that the design process is transparent, reproducible and useful as a resource to guide subsequent work. The purpose of the application is to facilitate the research process and the way of locating, filtering, and evaluating articles across numerous different sources with the aim to make their work more focused and productive. In addition to individualized value, the solution has been recognized as a market intelligence system that can provide sponsor value by helping to detect leads, improve engagement insights and make strategic decisions at an organizational level.

One of the major goals of the project is to emancipate manually based research process inefficiencies by not less than 60 percent, thereby increasing the productivity of process as well as enabling organization and individuals to realign their resources with more valuable functions. Furthermore, the adherence to data privacy laws and ethical AI development standards will be a priority in the creation of this application in order to make this system responsible and safe to use. With a technological innovation combined with the ethical aspects, the project is dedicated to the realization of a solution powerful and efficient yet trustworthy and sustainable. This system would eventually serve as an effective resource to the research community and organizations as a whole since it will ultimately transform the methodology of how articles are searched, gauged, and applied in the research-based contexts.

2. Risk Management

Effective risk management is critical to ensuring the success of this project, given its reliance on external data sources, advanced machine learning (ML) and natural language processing (NLP) models, and compliance with strict ethical and regulatory standards. This section outlines the team's approach to identifying, categorising, and mitigating risks throughout the project lifecycle.

2.1 ACS Code of Ethics Overview

The project team commits to upholding the **ACS Code of Ethics**, which emphasises:

- **Honesty:** Transparent communication with stakeholders, accurate documentation, and responsible disclosure of project limitations.
- **Trustworthiness:** Protecting sponsor, user, and organisational data, ensuring outputs are reliable and unbiased.

- **Fairness:** Designing ML/NLP models that avoid demographic or contextual bias, validated against diverse datasets.
- **Professional Development:** Ensuring continuous learning within the team to address unfamiliar technologies and improve outcomes.
- **Public Interest:** Respecting privacy laws (GDPR, CCPA, Australian Privacy Act), API terms of service, and accessibility standards (WCAG 2.1 AA).

These principles guide the identification of risks, their treatment, and the ongoing monitoring process.

2.2 Team Values in Risk Management

The team's values; **openness, collaboration, respect, and trust**, play an active role in risk mitigation. Open communication ensures risks are flagged early, collaboration allows multiple perspectives in finding solutions, respect ensures that differences are constructively handled, and trust fosters accountability when risks require urgent action.

2.3 Risk Identification and Categories

The following risk categories have been identified:

- **Technical Risks:** Issues related to APIs, ML/NLP performance, data quality, and integration across system components.
- **Ethical and Legal Risks:** Risks of bias in algorithms, inadvertent data privacy violations, or breaches of API licensing terms.
- **Operational Risks:** Misalignment between backend, frontend, and AI teams; schedule compression near deliverables; sponsor feedback delays.
- **Resource Risks:** Limited student availability due to academic workload, reliance on free-tier APIs, or outages in cloud services.
- **Reputational Risks:** Incorrect or irrelevant outputs reducing user trust, or publishing a system that fails accessibility standards.

2.4 Risk Matrix

Key

- **Likelihood/Impact:** L / M / H / VH
- **Level:** computed qualitatively (Low/Med/High/Extreme)

Risk Register (Matrix)

ID	Risk	Category	Likelihood	Impact	Level	Early Signals / Triggers	Mitigation (Preventive)	Contingency (Reactive)
R1	API rate-limit volatility / quota cuts	Technical / External	M	H	High	Spike in 429s; vendor policy emails; slowed fetch	Multi-provider fallback, caching of recent queries; staggered calls; monitor quotas	Temporarily degrade to cached results; pause affected source; notify sponsor
R2	Platform ToS changes / breach	Legal / Compliance	M	H	High	ToS update notices; blocked endpoints	ToS review before integration; whitelist endpoints; log access patterns	Disable offending calls; contact vendor; switch provider
R3	Use of unauthorised APIs or methods	Legal / Compliance	L	H	Med	Missing API keys; scraping endpoints without permission	Use only public/approved APIs; keep a source registry	Remove offending integration; replace with compliant source
R4	Query ambiguity (Apple/Jaguar)	Functional / UX / ML	H	M	High	Irrelevant top results; user filters overused	Entity linking; user disambiguation prompts (industry/location); domain filters	Escalate to manual label rules; hotfix query-expansion blacklist
R5	Bias in ML/NLP models	Ethical / Quality	M	H	High	Skewed outcomes by group/topic; audit flags	Diverse training data; bias tests; fairness metrics in CI	Retrain with balanced sets; suppress biased features
R6	Irrelevant results / low precision	Quality / ML	M	H	High	Low CTR; sponsor feedback; poor validation metrics	Hybrid scoring (lexical+semantic+engagement); threshold tuning; AB tests	Roll back model; increase manual rules; fast patch weights

R7	Performance/latency (large datasets)	Technical	M	M	Med	P95 latency > target; timeouts	Lightweight models for MVP (MiniLM); batch scoring; async queues; DB indexes	Add caching; precompute hot queries; temporarily reduce result size
R8	Cloud/tool dependency outage (GitHub, MongoDB, CI/CD)	Operational	M	M	Med	Vendor status red; failed pipelines	Local dev fallbacks; weekly offline backups; infra runbook	Switch to local DB; commit offline; re-run jobs after recovery
R9	Data privacy non-compliance (PII)	Legal / Compliance	L	VH	Extreme	PII detected in logs; user complaints	Data minimisation; PII detectors in pipeline; suppression list; DPIA checklist	Purge data; rotate keys; breach notification per policy
R10	IP / copyright on republication	Legal	M	H	High	DMCA/takedown notices; blocked embeds	Store metadata/snippets; link out; respect robots/no-republish terms	Remove content; switch to summaries/links; legal review
R11	Toxic/unsafe content exposure	Safety / UX	M	H	High	Profanity/hate in results; user reports	Content filters (blocklists, toxicity model); safe search flags	Quarantine items; refine filters; add "report" flow
R12	Accessibility non-compliance (WCAG 2.1 AA)	Compliance / UX	L	H	Med	Failing axe tests; QA findings	Design reviews by QA; automated a11y tests; keyboard/contrast checks	Triage fixes; a11y hotfix sprint; defer features that block compliance
R13	Integration drift (schema/API contracts)	Technical / Process	M	H	High	4xx/5xx between services; failing contract tests	Shared OpenAPI schema; contract tests in CI; versioned endpoints	Freeze API; patch adapters; post-mortem & update contracts

R1 4	Scope creep	Delivery	M	H	High	New features mid-sprint; backlog ballooning	MoSCoW prioritisation; change-control board; sponsor sign-offs	De-scope to MVP; defer to later milestone
R1 5	Sponsor feedback delays	Stakeholder	M	M	Med	PRDs pending; blocked approvals	Set SLAs (3–5 days); provide decision packets; early demos	Proceed with conservative defaults; flag risks in status report
R1 6	Schedule compression near D3/D4	Delivery	H	H	Extreme	Testing tasks squeezed; rising open bugs	Feature freeze 7 days pre-deadline; test/handover buffer week	Slip non-critical scope; add all-hands test day
R1 7	Student capacity/time conflicts	Resource	M	M	Med	Exams week; missed stand-ups	Load-leveling in sprint planning; smaller tickets; backups per task	Reassign tasks; cut non-essentials; extend hours short-term
R1 8	Knowledge silos (single-point failure)	Team / Process	M	H	High	Only one approver on module; blocked reviews	Pair programming; module READMEs; cross-reviews	Swarm on module; rotate ownership
R1 9	Honesty/communication lapses	Governance / Ethics	L	H	Med	Undisclosed blockers; late surprises	Daily updates; blocker flag within 24h; retro culture	Escalate to lead; reset plan; transparent sponsor note
R2 0	Security breach (keys, secrets)	Security	L	VH	Extreme	Secrets in repo; unusual traffic	.env/secret vault; least privilege; secret scanning; HTTPS everywhere	Revoke/rotate; incident response; audit & patch
R2 1	Version control/merge regressions	Quality / Process	M	M	Med	Broken main; flaky builds	Branch protection; required reviews; CI test gates	Hotfix branch; revert; post-mortem

R2 2	Source data quality (spam/low-signal)	Data Quality	M	M	Med	Many low-quality posts; user complaints	Source whitelists; spam filters; engagement heuristics	Down-rank source; manual curation list
---------	---------------------------------------	--------------	---	---	-----	---	--	--

Monitoring & Reporting (embed under the table if you have space)

- **Risk Register Location:** GitHub/Confluence (single source of truth)
- **Review Cadence:** Weekly team meeting; escalation within 48h for High/Extreme risks
- **Status Codes:** Open / Monitoring / Mitigating / Closed (update each sprint)

2.5 Monitoring and Reporting

- **Weekly Team Meetings:** Progress and risks are reviewed; new risks logged.
- **Risk Register:** Maintained in GitHub/Confluence with updates to status, owners, and mitigation actions.
- **Sponsor Engagement:** Risks impacting scope, compliance, or deliverables are flagged in fortnightly meetings.
- **Escalation Policy:** Unresolved risks are escalated to the sponsor or unit convenor within 48 hours.

3. Resource Management

3.1. People

Team values

To work effectively as a team, we agreed on a set of shared values that guide how we collaborate and support each other. Here are the following values we uphold:

1) **Openness:**

Openness is essential for keeping both the team and the sponsor aligned throughout the project. Research in project management highlights that transparency reduces misunderstandings and strengthens accountability. Given that the project spans frontend, backend and AI components, there is a risk of one area becoming disconnected from the others. Therefore, by sharing progress, delays and obstacles openly, the team ensures that potential issues are surfaced early and addressed collaboratively. For example, if the backend team encounters delays with API integration, openness allows the AI developers to adjust their testing timelines rather than waiting idly.

2) **Collaboration:**

Collaboration is one of the foundations of team building as it allows us to share ideas, support one another and solve problems faster. Studies in software engineering show that practices such as pair programming and peer code reviews lead to higher quality outcomes and fewer errors. In a project like ours, where multiple layers of technology have to work together, collaboration ensures we do not build in isolation. In practice, this means working together on tasks that cut across roles, such as designing APIs jointly between backend and AI or reviewing frontend code as a group to ensure consistency.

3) **Respect:**

Respect underpins everything we do and is critical for building a positive and productive working culture. Findings from Google's Project Aristotle emphasise that psychological safety, which is the ability to contribute without fear of judgment, is a key driver of team effectiveness. Our group includes members with different levels of expertise, so showing respect means recognising these differences while valuing everyone's contribution. Rather than dismissing questions, we aim to answer them clearly and supportively. For instance, if someone without an AI background asks for clarification on a model's output, the AI developers will explain the reasoning behind it in a way that helps the whole team learn.

4) **Trust:**

Trust is what allows us to move forward without second-guessing each other. Organisational studies show that teams with high trust spend less time monitoring and more time producing meaningful work. For us, this means once a task is assigned, we assume it will be delivered unless otherwise communicated. This helps with planning and coordination, as other members can build their own timelines around those commitments. For example, if the backend team commits to delivering endpoints by Friday, the AI team can plan to start integration immediately afterward, confident that the work will be ready. Trust, in this sense, allows us to work efficiently under tight deadlines.

Team structure

The project team is organised into clearly defined roles to ensure accountability, balanced workload and effective collaboration. Each member has been assigned responsibilities that align with their strengths and contribute to the overall success of the project.

Name	Role	Responsibility Highlight
Anirudh Gupta	Group leader	Oversees project direction and coordination
Khawaja Abeer	Technical lead	Guides technical decisions and architecture
Eric Kok	Documentation lead	Maintains meeting minutes

		and documentation
Yousef Diab	Quality assurance lead	Ensures deliverables meet quality standards
Amr Abdelghaffar	Quality assurance lead	Supports quality checks and validation
Edward Chairudin	Testing lead	Designs and manages testing processes

Roles and responsibilities

To ensure clarity and accountability, each role in the team has been matched with specific responsibilities based on expertise and project needs, as shown below.

Role	Responsibilities	Assigned To
Web and app developer	Frontend development, Django templating and UX/UI design	Yousef
Software tech developer	Backend logic, Django setup, API integration, database management, deployment	Eric, Edward
ML/AI developer	NLP model development, data processing, data scraping, topic relevance scoring	Anirudh, Khawaja, Amr

Stakeholders

Our project involves several stakeholders, each playing a different role in guiding, supporting and ultimately benefiting from the system. These can be grouped into internal and external stakeholders:

Category	Stakeholder	Role / interest
Internal	Project team members	Responsible for design, coding, testing and documentation to deliver the final product.
	Sponsor (Robotic Marketer)	Defines requirements, reviews progress and ensures the system

		supports business objectives.
External	End users	Use the system to access ranked and summarised articles for research/insight.
	Unit convenor & teaching staff	Ensure academic alignment, assess deliverables and provide guidance if conflicts arise.
	Compliance / legal advisors	Ensure ethical AI use, data privacy compliance and adherence to API licensing terms.

Communication pattern

1) Day to day communication

- Quick updates, reminders and coordination take place on WhatsApp and Discord. These channels are used for informal and immediate interactions among team members.

2) Weekly team communication

- The team holds weekly Discord meetings to review progress, assign new tasks and address any ongoing issues or obstacles.

3) Weekly sponsor check-ins

- Formal meetings with the sponsor are conducted on Microsoft Teams every week. These sessions are used to share updates, discuss potential challenges and gather feedback for alignment.

4) Documentation collaboration

- Google Docs is used for drafting, collaborative writing and maintaining technical documentation.

5) Code collaboration

- GitHub is also used for pull requests, code reviews and collaborative development, which ensures that version control and quality checks are consistently applied.

6) Service-level expectations for communication

- Internal team chats (WhatsApp/Discord): responses expected within 4 hours on weekdays.
- Sponsor communication (via email): responses expected within 24–48 hours.
- Critical blockers: flagged within 24 hours during stand-ups or directly in the group.

Conflict resolution strategies

Strategy 1

This approach focuses on resolving issues in progressive stages:

- 1) **Direct conflict:** If there is a conflict between team members, the first attempt is to resolve disagreements privately and respectfully.
- 2) **Team review:** If the conflict is still unresolved, the issue will be raised in the weekly team meeting for open discussion.
- 3) **Mediation:** If there is no majority vote, the team leader will step in to mediate and suggest a compromise.
- 4) **Sponsor intervention:** If the issue is still unresolved, the matter will be escalated to the sponsor for ultimate decision.
- 5) **Prevention step:** The team will reflect on the conflict and come up with an improved conflict resolution strategy for potential conflict in order to reduce the chance of a conflict escalating further.

Strategy 2

This approach emphasises proactive steps to avoid conflicts before they escalate:

- 1) **Clear role definition:** Ensure all members understand their roles and responsibilities to reduce overlap or misunderstandings.
- 2) **Shared values:** Reiterate team values (openness, collaboration, respect, trust) at the start of meetings to set the tone.
- 3) **Check-ins:** Hold quick mid-week check-ins to catch frustrations early.
- 4) **Documentation:** Keep transparent records of decisions and expectations in shared platforms (Google Docs, GitHub) so no one feels left out.
- 5) **Feedback culture:** Encourage constructive feedback to surface disagreements as “ideas to improve” rather than personal disputes.

3.2. Hardware, Software and Other

To deliver this project effectively, the team relies on a combination of hardware, software and collaborative tools. These resources support development, testing, version control and communication which ensures that both technical and organisational needs are met. Below is a breakdown of the key resources we will be using, grouped by category.

Hardware

- **Servers for AI model training:** Cloud-based servers capable of handling model training and inference workloads. These provide the required compute resources (CPU/GPU) for running NLP models.
- **Local development machines (Fallback strategy):** Each team member's laptop/PC serves as a backup in case cloud services become unavailable.

Software

- **GitHub:** Version control for both code and documentation, enabling branching, pull requests and collaborative reviews.
- **GitHub models:** Provides access to hosted pre-trained AL/ML models for quick prototyping and integration. As it reduces the need to train from scratch and accelerates development.

- **Hugging Face Transformers:** Pre-trained NLP models used for semantic similarity and text summarisation.
- **MongoDB Atlas:** Cloud-based NoSQL database for storing articles, metadata and processed outputs.
- **Django REST Framework:** Backend framework for API development and orchestration between data sources and AI components.
- **HTML/CSS/JavaScript:** Core web technologies for building and styling the frontend interface.
- **Large Language Mode:** Supports semantic analysis and generates concise article summaries.
- **Scikit-learn:** Library for TF-IDF and other machine learning utilities used in relevance ranking.
- **Figma:** Collaborative design tool for creating and iterating on UI wireframes.
- **Playwright:** End-to-end testing framework ensuring frontend reliability and consistent behaviour across browsers.
- **Jira:** Project management tool for sprint planning, backlog tracking and Scrum practices.
- **Confluence or GitHub Wiki:** Used to document technical decisions, processes and knowledge sharing.

Other resources

- **API access to external sources:** Access keys and rate-limited connections to platforms like Reddit, LinkedIn and news APIs for fetching articles.
- **Communication tools:** WhatsApp for day-to-day updates, Teams for sponsor meetings and Discord for weekly check-ins.

4. Change Management

4.1. Managing requirements and scope change

Documentation of Changes:

Given the ever-changing nature of the project, the team must maintain a clear procedure to manage changes in the plan. Any changes to the requirements or project scope will first be documented clearly, ensuring that no sponsor requests are overlooked, and to also allow all team members to have access to the information. The team will then ask for any needed clarifications on the details regarding the sponsor request, ensuring there is no ambiguity or misunderstanding while the plan is being updated. Once confirmed, the project plan will be updated to reflect the changes.

Assessment of Timeline Impact:

Each change will be assessed for its impact on the project timeline while the plan is being updated. If necessary, the existing timeline will be updated to accommodate for the changes, ensuring the plan continues to be realistic and achievable within the required deadline. A change request log will then be made, recording the date, description of change, the approval status, and the owner responsible for managing the change. A two-day approval policy will be applied for sponsor approval of the updated plans made, ensuring no unnecessary delays are created.

Prioritisation of Change Requests:

To appropriately prioritise the change requests, the MoSCoW method will be used, requiring all changes are classified as “Must Have”, “Should Have”, “Could Have”, or “Won’t Have”, with advice from the sponsor. This ensures both the team and sponsor have a shared understanding of the relative importance of each change request. Additionally, for change requests that involve affecting the application’s interface, cross browser compatibility must be reviewed before the change is implemented, to ensure alignment with existing standards.

Integration into Agile Workflow:

After finalising the change planning and approval process, the change should be mapped to a Jira issue, with linked pull requests. Then the Jira issue should be assigned to either the current sprint week, or deferred to the next sprint if the approval process had been completed after day 3 of the 1-week sprint. This ensures that the changes made do not potentially lead to delays in delivering the current sprint’s goals.

By following this structured process, the project team can ensure that changes are managed transparently, whilst meeting sponsor expectations, reducing the risk of miscommunication, overlooked changes, or delays to the project.

4.2. Version Control

Version control will be applied to documents and data alongside the code. GitHub will be used as the central repository for managing all files, providing transparency and accountability across the entire project. The team will adopt consistent branch naming conventions and require all pull requests to be reviewed before any changes are merged into the main branch. Each version of the project will also include release tags, using semantic versioning aligned to the deliverable each file is for. This ensures that all updates are peer-reviewed, reducing the chance of errors being introduced to the project, improving the overall quality.

To maintain accuracy and professionalism in both code and documentation, quality leads will review code and documents prior to submission. This process provides an additional opportunity for inaccuracies or errors to be found prior to submission. By using GitHub, the team will minimise the risk of version conflicts, maintain clear records of all updates and their author, improving the team’s ability to work collaboratively.

5. Quality Management

Maintaining the platform’s quality is essential to ensure it delivers relevant and accurate information. Since the application sources data from external data sources, quality management will focus on: Data integrity, ML model performance and system reliability. A structured quality control plan will be adopted to monitor, evaluate and continuously improve the project deliverables.

- **Document Quality:** All project documents (Team manuals, feasibility report, project plan, etc) will be reviewed by the Quality Assurance Leads before submission

- **Progress Monitoring:** Weekly team meetings are scheduled to ensure deliverables are met promptly and risks are detected early.
- **Test Planning:** The Testing Lead will design test cases covering data extraction, ML relevance scoring, and UI output to validate end-to-end functionality.
- **Sponsor Engagement:** Fortnightly meetings with the sponsor to ensure sponsor feedback on progress and alignment of expectations.
- **Code Assurance:** GitHub branch protection and mandatory code reviews will safeguard code quality, version control, and maintainability
- **Team Communication:** Open discussions will be encouraged to raise issues early and collaborate on solutions.
- **Agile Review:** Sprint reviews at the end of each cycle enable the team to evaluate progress, identify areas for improvement, and refine upcoming tasks.
- **Automated Testing:** Continuous Integration pipelines will include automated unit, integration, and end-to-end testing to ensure robustness of keyword search, data retrieval, and ML scoring components.

5.1. Managing Change

Given the evolving nature of external data sources (Reddit, linkedin, X, etc.) and machine learning models, changes to the project are expected. To manage change effectively:

- A change Log will be maintained on GitHub, which contains all the change requests.
- Each change will be evaluated based on its impact on scope, timeline and quality.
- Changes to a new ML Model will require testing and validation before adoption.
- Approval of major changes will require discussion in weekly meetings and confirmation from the sponsor.
- Minor changes (e.g., bug fixes, UI tweaks) may be approved directly by the Quality Lead.
- Any accepted changes will be documented and reflected in the sprint backlog, ensuring traceability.

5.2. Conflict Resolution

Team-based projects often involve differing opinions, particularly around technical decisions such as ML model selection, feature prioritisation, or testing approaches. To prevent conflicts from affecting progress:

- **Early identification:** Team members are encouraged to raise concerns immediately during stand-ups or weekly meetings.
- **Open discussion:** Conflicts will first be addressed collaboratively within the team to find a compromise
- **Escalation:** If no agreement is reached, the issue will be escalated to the Project Lead for resolution.
- **Sponsor involvement:** For decisions impacting project scope or deliverables, the sponsor's feedback will be sought.
- **Documentation:** Outcomes of conflict resolution will be recorded in meeting notes to ensure clarity and accountability.
- **Convenor involvement:** For major disruptions, a dialogue will be opened with the convenor to seek a solution.

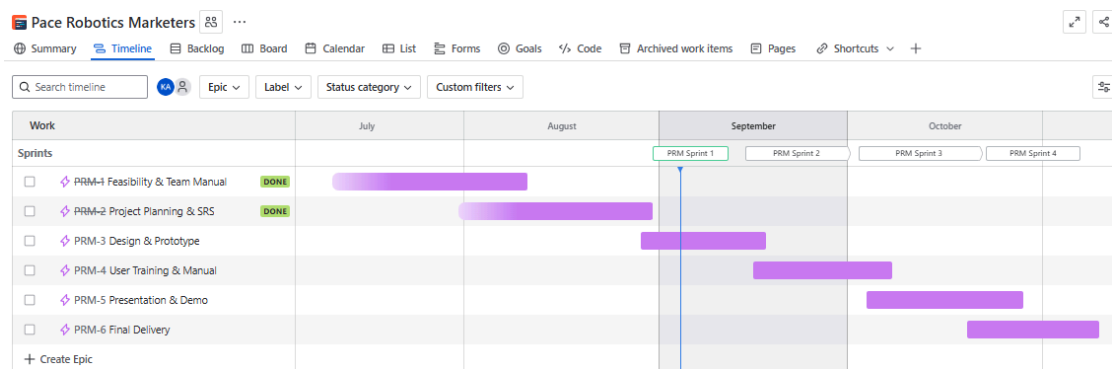
Any decision requires a vote from the entire team to be approved. If a resolution is not reached, the final say lies with the Team Leader.

6. Project Schedule

Every week we try to work in sprints. Before a deadline we stop adding new things, test, fix, and polish. The checklist includes testing if every automated test possibly passes, if there any blocking bugs may be around, and if it runs on Chrome, Edge, Safari. Changed docs updated, demo runs end-to-end, and approvals recorded.

6.1. Jira for Project Management

Project Timeline:



Task breakdown:

Q Search list		 Filter											
☐	Type	Key	↑	Summary	Status	Comments	Assignee	Due date	Priority	Labels	Created	Updated	Reporter
☐		PRM-1		Feasibility & Team Manual	DONE	 1 comment		 Aug 10, 2025	High	deliverable	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-2		Project Planning & SRS	DONE	 1 comment		 Aug 30, 2025	High	deliverable	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-3		Design & Prototype	IN PROGRESS	 1 comment		 Sep 17, 2025	High	deliverable	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-4		User Training & Manual	TO DO	 1 comment		 Oct 7, 2025	Medium	deliverable	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-5		Presentation & Demo	TO DO	 1 comment		 Oct 28, 2025	Medium	deliverable	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-6		Final Delivery	TO DO	 1 comment		 Nov 9, 2025	High	deliverable	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-7		Write Feasibility Report Section 8	DONE	 1 comment		 Aug 5, 2025	High	report	 Aug 28, 2025	 Aug 31, 2025	 Khawaja Abeer Ashraf
☐		PRM-8		Add Alternative Solution 4.3	DONE	 1 comment		 Aug 7, 2025	High	report	 Aug 28, 2025	 Aug 31, 2025	 Khawaja Abeer Ashraf
☐		PRM-9		Complete Team Manual Section 4	DONE	 1 comment		 Aug 8, 2025	Medium	manual	 Aug 28, 2025	 Aug 31, 2025	 Khawaja Abeer Ashraf
☐		PRM-10		Draft Project Plan Risk Management	DONE	 1 comment		 Aug 20, 2025	High	plan	 Aug 28, 2025	 Aug 31, 2025	 Khawaja Abeer Ashraf
☐		PRM-11		Prepare SRS Section 2.1-2.4	DONE	 1 comment		 Aug 25, 2025	High	srs	 Aug 28, 2025	 Sep 2, 2025	 Khawaja Abeer Ashraf
☐		PRM-12		Prepare SRS Draft for sponsor feedback	DONE	 1 comment		 Aug 31, 2025	High	srs	 Aug 28, 2025	 Sep 2, 2025	 Khawaja Abeer Ashraf
☐		PRM-13		Set up Backend API	IN PROGRESS	 1 comment		 Sep 20, 2025	High	backend	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-14		Implement NLP Preprocessing	TO DO	 1 comment		 Sep 22, 2025	High	nlp	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-15		Build Relevance Model	TO DO	 1 comment		 Sep 24, 2025	High	nlp	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-16		Write User Manual Draft	TO DO	 1 comment		 Oct 15, 2025	Medium	docs	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-17		Prepare Training Material	IN PROGRESS	 1 comment		 Oct 17, 2025	Medium	docs	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-18		Prepare Slide Deck	TO DO	 1 comment		 Oct 28, 2025	Medium	presentation	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-19		Demo Application	IN PROGRESS	 1 comment		 Oct 30, 2025	High	demo	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-20		Handover Documentation	IN PROGRESS	 1 comment		 Nov 10, 2025	High	handover	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
☐		PRM-21		Final Sponsor Review	TO DO	 1 comment		 Nov 12, 2025	High	handover	 Aug 28, 2025	 Sep 4, 2025	 Khawaja Abeer Ashraf
+ Create													

Release Plan:

Stage (timeline)	Key tasks & processes	Deliverables	Allocated to (owners)	Resources (tools/env)
------------------	-----------------------	--------------	-----------------------	-----------------------

Week 5 – Internal Alpha	Create Django app; prove MongoDB 8+ connection; add /api and health check; draft Search & wireframes; basic repo setup	Running dev app; README/setup notes; wireframes v1	Eric, Edward (API/DB), Yousef (UI)	Python 3.12+, Django 5.2+, MongoDB 8+ (Atlas), PyMongo, .env.example, GitHub
Week 6 – Documentation Pack (Deliverable 2)	Finish Project Plan, SRS, Team Manual, Feasibility drafts; stabilize base API routes; revise wireframes	Submitted D2 docs; stable base API & route list	All (section owners), Yousef (frontend notes)	Repo docs, meeting minutes, MS Teams, GitHub
Weeks 7–8 – Scoring MVP	Implement lexical + semantic scoring; build server-rendered Search & History pages; add pagination + loading states; simple keyboard navigation	Working results list; basic scoring; screenshots; test cases (happy path)	Anirudh, Amr, Khawaja (scoring), Yousef (pages)	Open-source NLP libs, Django templates, PyMongo, Python 3.12
Week 9 – Prototype	End-to-end path input → ranking → output; error/empty states; simple logs for failures; prepare demo	Demo-ready build; demo script; short bug list	All	Same stack; slide outline; screen recorder
Week 10 – Hardening	Fix bugs; cross-browser checks (Chrome/Edge/Safari); keyboard-only path (skip link, visible focus); small speed improvements	Test summary; updated docs; fixed UI issues	Yousef (UI/a11y), Eric, Edward (API), AI team (scoring fixes)	Browsers on desktop/mobile, checklists, GitHub Issues

Week 11 – Docs & Training	User/training notes; operations runbook (start/stop, env vars); improve tests; light refactor for clarity	User/training doc; runbook; updated tests	Eric (docs/runbook), Yousef (UI guide), All (reviews)	Repo docs, README/runbook templates
Week 12 – Presentation	Polish demo; deck + script; rehearse; apply feedback	Polished demo; slide deck; rehearsal notes	All (presenters), Yousef (UI walkthrough)	Slides, demo dataset, meeting room/Teams
Final – Handover	Package code, docs, runbook; credentials checklist; sponsor sign-off	Final handover pack; sign-off record	Leads (handover), All (checks)	GitHub release/export, checklist template

7. Assumptions

To pull off this project successfully, a number of key assumptions have been made concerning resources, tools, processes and team commitments. It is assumed that the current data retrieval APIs and datasets will continue to be accessible at their existing level and cost and that there will be no major alterations in API provider terms of service. It can be supposed that all the team members should have the skills necessary to fulfill their tasks and be available during the planned weekly meetings every Friday at 3 PM, as well as during two to three ad hoc check-ins, whenever needed. It is also assumed that GitHub Projects will be used systematically to manage progress, where the tasks are divided into small chunks that can be tested easily, and that services necessary to the project, such as MongoDB Atlas, Fly.io and GitHub Actions, will be available over the lifespan of the project.

It is also possible to use tools like the Playwright and Figma that should be compatible with the Django deployment, and the stable availability of the internet and the cloud services are assumed to be provided during the project duration. The sponsor should be able to give timely feedback within three to five days so as to keep the progress in order. It is further assumed that everyone is capable of equal contribution to workload, although they may need adjustment during exam and assignment time when they may be less available. Lastly, the project assumes that the various tools and platforms needed in the development of the project will remain free and open source in order to guarantee the accessibility and sustainability of the same.

8. Standards/Templates/Appendices/Forms

8.1 Coding & configuration standards.

- Python 3.12+, Django 5.2+; follow PEP 8. Use semantic HTML and plain CSS (no framework).
- Configuration via environment variables; no secrets in source control.

8.2 Version control & reviews.

- GitHub with branch protection; all changes via Pull Request.
- Conventional Commits for clear history; at least one reviewer approval before merge.

8.3 Documentation standards.

- SRS, Project Plan, Team Manual, and Feasibility kept in version control and updated per release.
- Architecture Decision Records (ADRs) for significant technical/process choices.
- Meeting minutes captured for all client/team meetings and stored with the project docs.

8.4 Testing & quality gates.

- Unit/integration tests for backend functions; basic E2E coverage for main user flows.
- Accessibility basics: keyboard-only operation, skip link, visible focus, sensible tab order.
- Cross-browser smoke: Chrome, Edge, Safari (desktop/mobile).
- CI must pass before merge; no open P1 defects at release time.

8.5 Security & privacy practices.

- HTTPS for all traffic; input/output validation and sanitization.
- API keys/credentials remain server-side (MONGODB_URI); logs minimized and exclude PII.
- Respect external platform Terms of Service and applicable privacy obligations.

8.6 Accessibility & UX standards.

- Aim for WCAG 2.1 AA: clear headings/labels, focus visible, link purpose clear, skip-to-content, logical focus order.
- Consistent layout and terminology across Search and History screens; clear empty/error states.

8.7 Templates (provided with the repo/docs).

- Meeting minutes, weekly report, change request, risk register, test case sheet, bug report, release notes, ADR template, handover checklist.

8.8 Appendices.

- API & Data Source Register (endpoints, limits, ToS).
- Glossary (project terms and acronyms).
- License & Attribution Register (libraries/datasets).
- Operations Runbook (start/stop, environment variables, backups).

8.9 Forms.

- Change Request Form (scope, rationale, impact, approval).
- Issue/Bug Form (steps to reproduce, expected/actual, severity).

- Risk Log Entry (risk, likelihood/impact, owner, mitigation).
- Release Sign-off (checklist, approvers, date/tag).

Project Progress

Provide a summary (up to one-page) that describes what the team have done so far related to the project, which is additional to the Feasibility Report, Team Manual, Project Plan and Requirements/Scoping Document. Consider the following and use these headings:

- **Research and Training** – what has been investigated? What knowledge and skills gaps have been identified? How will they be filled? What training/preparation has been done.

The team has undertaken extensive research into the technical landscape of the project, focusing on data retrieval methods, API integration, and machine learning techniques for article ranking and summarisation. Investigations have included comparing different NLP models (e.g., transformer-based LLMs vs. lightweight classifiers) and exploring database options such as MongoDB Atlas for scalable storage.

Knowledge gaps identified include advanced handling of API rate limits, ensuring WCAG 2.1 AA accessibility compliance, and optimising AI models for performance under limited resources. To address these, the team has engaged in self-directed training through Hugging Face tutorials, GitHub APIs, Django REST documentation, and collaborative peer-learning sessions.

Initial trial runs of preprocessing pipelines and summarisation models have provided practical experience, reducing uncertainty around implementation.

- **Project Tasks and Activities** – What has been started/completed so far?

Beyond the feasibility report, team manual, project plan, and scoping document, several practical activities have been initiated. A Jira Scrum board has been established to manage sprints and track progress, with tasks allocated to backend, frontend, and AI teams.

Initial preprocessing pipelines have been prototyped, focusing on cleaning and structuring articles from Reddit and LinkedIn APIs. Draft wireframes of the frontend interface have been produced using Figma, aligning with accessibility guidelines. The SRS has been collaboratively developed, detailing external interface requirements and system features. Sponsor meetings have provided clarification on requirements and confirmed the priority of features such as relevancy scoring and semantic summarisation.

- **Testing, Verification and Validation** – what testing, V & V will be needed? What has been done so far? What feedback has been received and how has/will it be acted upon?

Testing is planned to occur at multiple levels: unit tests for API calls, integration tests for data pipelines, and validation of relevancy scoring through both automated metrics and sponsor feedback. Verification will include checking that “shall” requirements in the SRS are implemented and traceable, while validation will focus on ensuring the system meets user and sponsor expectations. So far informal testing of preprocessing steps has been conducted to confirm that API calls retrieve and parse data correctly without breaching provider ToS.

Feedback from the sponsor has emphasised the need for explainability in scoring and compliance with data privacy laws; this has been integrated into the project plan by including PII detection

mechanisms and bias mitigation strategies. Automated testing pipelines are also being prepared in GitHub Actions to ensure continuous validation throughout development.

NOTE: This template is shareware downloaded from **HYPERLINK** "<http://www.processimpact.com>" www.processimpact.com. All shareware payments are donated to the Norm Kerth Benefit Fund to help a consultant who is disabled with a brain injury. Please visit **HYPERLINK** "http://www.processimpact.com/norm_kerth.html" http://www.processimpact.com/norm_kerth.html to make a shareware payment (\$10 suggested). Thank you!

Software Requirements Specification

for

AI-Powered Web Application for Bot Listening

Version 1.0 approved

Prepared by Team 22

Robotics Marketers Pty Ltd

04/09/2025

eLearning versions of several popular Process Impact training seminars are available at **HYPERLINK** "<http://www.processimpact.com/elearning.shtml>" www.processimpact.com/elearning.shtml, including "In Search of Excellent Requirements," "Exploring User Requirements with Use Cases," "Writing High-Quality Requirements," "Software Inspections and Peer Reviews," and "Project Management Best Practices". Single-user and corporate-wide site licenses are both available.

Table of Contents

1. Introduction	4
1.1 Purpose	4
1.2 Document Conventions	4
1.3 Intended Audience and Reading Suggestions	5
1.4 Project Scope	5
1.5 References	6
2. Overall Description	6
2.1 Product Perspective	6
2.2 Product Features	7
2.3 User Classes and Characteristics	9
2.4 Operating Environment	10
2.5 Design and Implementation Constraints	10
2.6 User Documentation	10
2.7 Assumptions and Dependencies	11
3. System Features	11
3.1 System Feature 1 - Article retrieval	11
3.2 System Feature 2 - Relevance Scoring & Ranking	12
3.2.1 Description and Priority	12
3.2.2 Stimulus/Response Sequences	12
3.2.3 Functional Requirements	12
3.3 System Feature 3 - Filtering, Disambiguation & Facets - elaborate	12
3.3.1 Description and Priority	12
3.3.2 Stimulus/Response Sequences	12
3.3.3 Functional Requirements	13
3.4 System Feature 4 - Summarization (Optional/Configurable)	13
3.4.1 Description and Priority	13
3.5 System Feature 5 - Caching & Resilience	13
3.5.1 Description and Priority	13
3.5.2 Stimulus/Response Sequences	13
3.5.3 Functional Requirements	13
4. External Interface Requirements	14
4.1 User Interfaces	14
4.1.1 UI Principles & Compliance	14
4.1.2 Core Screens	14
4.1.3 Interaction & Usability	15
4.1.4 Context Diagram	15
4.1.5 User Interface Design	15
4.2 Hardware Interfaces	16
4.3 Software Interfaces	16
4.3.1 Internal Components	16
4.3.2 External APIs & Data Sources	16
4.3.3 Data Contracts	16
4.4 Communications Interfaces	17
	41

4.4.1 Transport & Formats	17
4.4.2 Authentication & Authorization	17
4.4.3 Reliability, Errors, and Rate-Limiting	17
4.4.4 Performance Targets	17
4.4.5 Security & Privacy	17
4.4.6 Observability	18
4.4.7 Traceability Considerations	18
5. Non Functional	
5.1 Performance Requirements	18
5.2 Safety Requirements	19
5.3 Security Requirements	19
5.4 Software Quality Attributes	19
Other Requirements	20

Revision History

Name	Date	Reason For Changes	Version
Eric	28/08/25	Added ideas in bullet points as a draft for sections 1to 4	1.0
Anirudh	28/08/25	Added bullet points as draft for all sections	1.0
Amr	28/08/25	Added bullet points as draft in sections 1 and 2	1.0
Khawaja	29/08/25	Added bullets in all sections for draft	1.0
Yousef	29/08/25	Added draft bullet points throughout the document	1.0
Eric	30/08/25	Added a sequence diagram in section 2.1	1.0
Eric	30/08/25	Added a data flow diagram in section 2.2	1.0
Yousef	30/08/25	Created UI designs screens	1.0
Khawaja	30/08/25	Added 4 functional requirements	1.0
Edward	30/08/25	Add some bulletpoint	1.0
Anirudh	30/08/25	Completed section 1.1 of the report	1.0
Edward	1/09/25	Do the 5.3 section	1.0
Anirudh	1/09/25	Completed section 1.2 of the report	1.0
Anirudh	1/09/25	Completed section 1.3 of the report	1.0
Khawaja	01/09/25	Added context diagram and UI designs	1.0
Eric	02/09/25	Finalised and refined section 2,1, 2.2 and 2.3	1.0
Khawaja	02/09/25	Finalised my assigned sections	1.0
Yousef	02/09/25	Completed section 5.1 and 5.2	1.0
Anirudh	02/09/25	Completed section 1.4 of the report	1.0
Edward	02/09/25	Do the 5.4 and 6 section	1.0
Amr	02/09/25	Completed sections 2.4 and 2.5	1.0
Edward	04/09/25	Fix the formatting of my section	1.0
Yousef	04/09/25	Revised the overall document before submission	1.0

Anirudh	04/09/25	Incorporated sponsor feedbacks in the report	1.0
Anirudh	04/09/25	Formatted section 1 for submission	1.0
Amr	04/09/25	Completed Section 2.6 and 2.7	1.0
Amr	04/09/25	Adjusted Section 2.4, 2.5, 2.6, and 2.7 to account for sponsor feedback	1.0

1. Introduction

1.1 Purpose

This software requirements specification document(SRS) defines the requirements for the development of an AI-powered research application, sponsored by Robotic Marketer Pty Ltd. This document specifies the requirements for version 1.0 (Initial MVP release).

The product proposed by Robotic Marketer is designed to assist users in retrieving, filtering and ranking articles from multiple online sources. The sources range from platforms like Linkedin and reddit to news sites and forums. This will be through the integration of third party APIs while keeping intellectual property rights and other licensing concerns in mind. The articles thus retrieved will be filtered and ranked by implementing ML techniques. By automating search and relevance scoring, the application addresses inefficiencies in manual research processes and provides faster access to high-value insights.

The scope of this SRS covers the following core features for Version 1.0:

- Automated retrieval of articles from approved sources via APIs.
- Relevance scoring and ranking of search results using ML algorithms.
- Summarization of retrieved articles for users to review the articles.
- A web-based interface that enables users to search, filter, and view ranked results.

This SRS applies to the full system and not a subsystem. It is intended to guide developers, project managers, testers, and stakeholders in aligning on system scope, functionality, and constraints for the initial release.

1.2 Document Conventions

This document follows the IEEE SRS standard format and uses the following conventions for clarity and consistency:

- Numbering - Sections and subsections are numbered hierarchically (e.g., 1.1, 1.2, 2.1.1) for ease of reference.
- Requirement Labels - Each requirement is assigned a unique label (e.g., REQ-1, REQ-2) to support traceability during development and testing.
- Priority Keywords - The following terms are used consistently:
 - Must - Mandatory requirement
 - Should - Recommended but not required
 - May - Optional
- Formatting Rules -
 - **Bold** text is used for requirement identifiers and key terms.
 - *Italics* are used for priority labels (*must*, *should*, *may*).
 - Merriweather font is used for code snippets or API references.

- Data and User References - References to requirements, data elements or figures use consistent labels (e.g., D1 = Search, D2 = Articles, D3 = Scores).
- Cross References - Internal references use section numbers and requirement IDs, while external references are listed in Section 1.5 (References).

1.3 Intended Audience and Reading Suggestions

This document is intended for the following stakeholders:

- Project Sponsor (Robotic Marketer Pty Ltd) : Project Sponsors confirm that the system requirements align with business objectives and expected outcomes.
- Project Manager : To plan development activities, allocate resources, and track progress against defined requirements.
- Developers : To design and implement system functionality in accordance with the specified requirements.
- Testing and Quality Leads : To derive test cases and ensure that functional and non-functional requirements are met.
- End Users : To understand the core functionality of the system and validate whether it meets their needs.
- Sponsor's technical staff: To evaluate system compatibility with hosting environments, databases, and security policies.

Here is how the document is intended to be read:

- Stakeholders seeking a high-level overview should review Section 1 (Introduction) and Section 2 (Overall Description).
- Developers and technical teams should focus on Section 3 (System Features) and Section 4 (Interfaces).
- Testers should prioritize Section 3 (Functional Requirements) and Section 5 (Quality Attributes).
- Project managers and sponsors may benefit from reviewing all sections, with emphasis on scope, assumptions, and dependencies.

1.4 Project Scope

Research analysts and knowledge workers spend significant time manually gathering, filtering, and ranking information. This inefficiency leads to lost productivity, inconsistent results, and slower decision-making. Robotics Marketer Pty Ltd is developing this AI-powered research application to automate these processes and deliver faster, more reliable insights. The key benefits of this project are:

- Efficiency: Reduce time spent on manual searching and filtering by automating relevance scoring and summarization.
- Consistency: Provide standardized ranking of sources, removing variability in human judgment.
- Scalability: Enable research processes to expand across industries, topics, and geographies without proportional increases in effort.
- Innovation: Position Robotics Marketer as a leader in AI-driven productivity tools, strengthening its competitive advantage.

Here is how this benefits for Businesses and Users:

- Businesses: Gain measurable productivity improvements, cost savings from reduced research time, and better decision-making through access to timely insights.
- Users: Save hours of daily effort, access relevant information quickly, and focus more on analysis rather than information gathering.

In Version 1.0, the initial MVP release, we have the following points in scope:

- Retrieval of articles from approved sources via APIs (e.g., Reddit, X, LinkedIn).
- Relevance scoring and ranking of results using ML.
- Summarization of selected articles.
- Web-based interface with search, filter, and browsing features.
- Secure data handling and compliance with privacy regulations.

1.5 References

- *IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications*. Institute of Electrical and Electronics Engineers, 1998.
- *WCAG 2.1 AA Accessibility Guidelines W3C Recommendation*, 06 May 2025. <https://www.w3.org/TR/WCAG21/>
- *Django REST Framework Documentation*. Version 3.15, Django REST Framework Documentation. Version 3.15, <https://www.django-rest-framework.org/>
- *GDPR Regulations (EU)* <https://gdpr-info.eu/>
- *Australian Privacy Act 1988 (Cth)*

2. Overall Description

2.1 Product Perspective

The product was proposed as part of the PACE project collaboration with Robotic Marketer, with the goal of extending the company's existing dashboard rather than creating a separate tool. Its primary role is to provide more efficient ways to perform market intelligence and social listening which allows users to automatically retrieve, rank, and summarise content from multiple online platforms. Instead of replacing any existing system, this product is a new module within the Robotic Marketer ecosystem, designed to work alongside analytics and reporting features already available on the dashboard.

The system architecture follows a three layer design, a frontend interface that delivers a web based user interface for keyword search and result display, a backend API which utilise Django REST framework that manages requests, handles multi-source article retrieval and connects with external sources, and an NLP/ML engine that performs relevance scoring, semantic similarity checks and automated summarisation. To support these functions, the product interacts with external APIs such as Reddit and news feeds as primary data sources, with strict adherence to their usage and compliance policies. Additionally, a centralised database persists fetched articles and metadata (including titles, URLs, sources and engagement metrics) in order to enable efficient querying, ranking, caching and reuse of results :

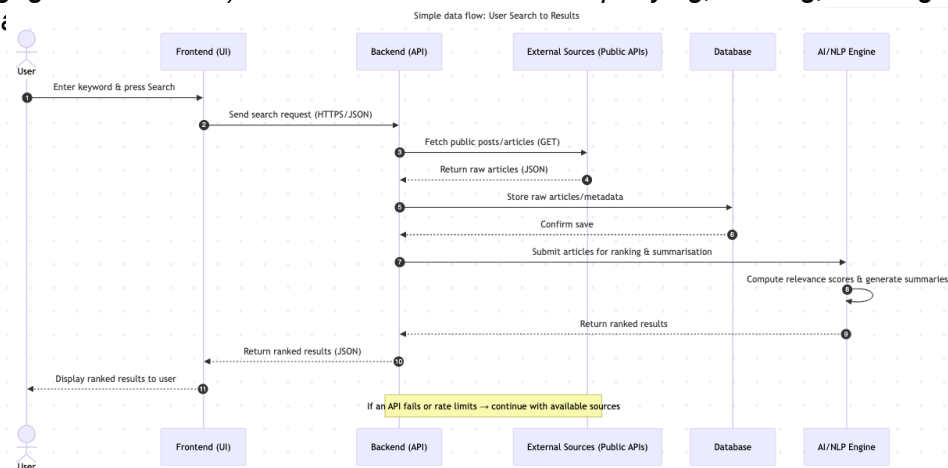


Figure 1: Data flow from user search to results.

The sequence diagram figure 1 above illustrates the end-to-end of the system. First, the user submits a keyword through the frontend interface, which sends the request to the backend API. The backend then fetches articles from multiple external sources, stores them in the database and forwards the data to the AI/NLP engine for ranking and summarisation. The AI component computes relevance scores and generates concise summaries before returning the ranked results to the backend. Finally, the backend then delivers the processed results to the frontend user interface, where they are displayed to the user. Importantly, if one of the external data sources fails or rate limits, the system continues to return results from the remaining available sources.

2.2 Product Features

This product is a feature within the Robotic Marketer dashboard that provides intelligent article retrieval and analysis. These are the main features the system will offer:

- **Keyword search interface**
Users can enter a keyword into the search field to begin their query. This acts as the starting point of the system which allows users to access multiple connected data sources with a single input.
- **Multi-source article retrieval**
The system collects articles and posts from multiple platforms such as Reddit and online news sites. By pulling data from different sources, users receive a more diverse perspective rather than relying on a single platform.
- **Relevance ranking**
Articles are automatically ranked using a hybrid scoring model that considers lexical similarity, semantic similarity and engagement signals. This ensures the most relevant and useful results are prioritised at the top of the list.
- **AI-powered summarisation**
Each retrieved article is paired with a short AI-generated summary. These summaries give users a quick overview of the content which helps them to decide at glance whether an article is worth reading in full.
- **Resilience to API failures**
If one or more external data sources encounter errors or rate limits, the system continues to return results from the remaining sources. This feature ensures consistent access to information without disruption.

The data flow diagram figure 2 below illustrates how each major feature of the system works together to deliver results to the user. The process begins when a user enters a keyword in the **keyword search interface**. The input is validated, normalised and saved as a search context before being passed to the backend.

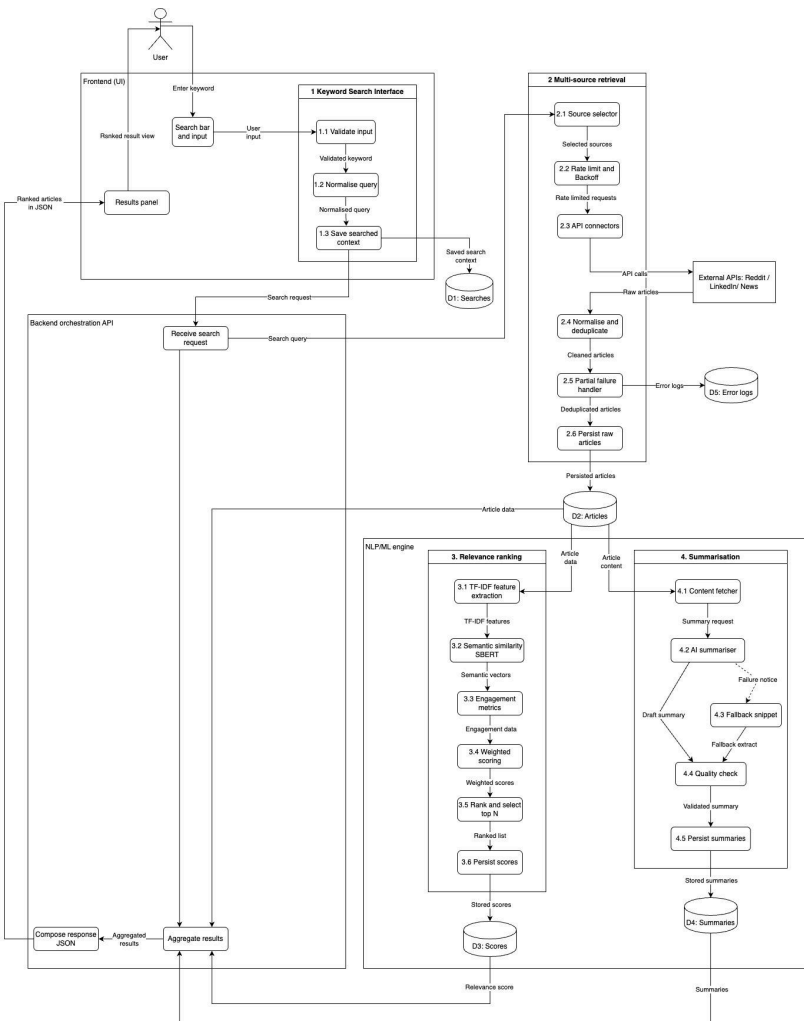


Figure 2: The relationship of each major feature.

From there, the backend orchestrates **multi-source article retrieval**, selecting appropriate sources, managing rate limits and connecting to external APIs such as Reddit, LinkedIn and news sites. Raw articles are cleaned, deduplicated and stored in the database, while any errors are logged. Next, the stored articles feed into the **relevance ranking** engine, which applies TF-IDF feature extraction, semantic similarity models like SBERT and engagement metrics to produce weighted scores. The ranked list is saved and made ready for display.

In parallel, the **summarisation** module processes the same articles, generating short AI summaries. If the summariser fails, a fallback snippet is provided. Each summary is checked for quality before being stored in the database. Finally, the backend aggregates ranked scores, summaries and article content into a single response, which is passed back to the frontend. The results are presented to the user in a clear and organised format which ensures that even if some sources fail, the system still provides reliable and useful output.

2.3 User Classes and Characteristics

The system is designed to support a range of user groups with different needs, levels of expertise and usage frequency. The following categories outline the primary, secondary and less critical user classes, along with their key characteristics and requirements.

Primary user classes (High priority - Favoured)

- Research analyst/ Market analyst
 - Frequency of use: Daily.
 - Technical expertise: Moderate to high, but prefers efficiency over technical detail.
 - Characteristics: Require accurate ranking, summarisation and freshness of data.
 - Requirement: The system must emphasise speed, clarity and reliability.
- Academic researchers
 - Frequency of use: Frequent (for literature reviews, case studies, coursework).
 - Technical expertise: Mixed, ranging from novice to advanced.
 - Characteristics: Need simple, intuitive UI to accommodate varying skills.
 - Requirement: Fast access to relevant and summarised content to reduce research time.
- Business decision makers / Sponsors
 - Frequency of use: Occasional.
 - Technical expertise: Low to high, may vary.
 - Characteristics: Expect polished presentation, reliability and minimal training curve
 - Requirement: Prioritise clarity, concise summaries and professional layout over technical depth.

Secondary user classes (Mid priority - Important)

- Data scientists / AI engineers
 - Frequency of use: Occasional to moderate.
 - Technical expertise: High.
 - Characteristics: Seek transparency in scoring models, fairness metrics and ranking logic.
 - Requirement: Explanations of weighting and relevance scoring to validate outputs.
- Knowledge workers (journalists, consultants)
 - Frequency of use: Occasional.
 - Technical expertise: Moderate.
 - Characteristics: Use tools for quick topic exploration and insights.
 - Requirement: Need fast, simple results without heavy configuration.

Less critical user classes (Low priority - Supplementary)

- Casual users (general public, content creator, bloggers)
 - Frequency of use: Rare, exploratory.
 - Technical expertise: Low.
 - Characteristics: Curious, non-professional usage.
 - Requirement: Lightweight interface with minimal learning curve.

2.4 Operating Environment

The system will be deployed as a cloud-hosted web application, as a part of Robotic Marketer's existing platform. It will operate across all standard client devices and modern browsers, while the backend will utilise the latest versions of various technologies, to ensure maintainability, and scalability. The following outlines the specific components of the operating system:

- **Client-Side:**
 - Accessible via any modern browser, such as Chrome, Firefox, or Edge.
 - Must support mobile browsers, with a fully responsive interface.
- **Server-Side:**
 - Developed in Python 3.12 or higher.
 - Uses the Django 5.2+ REST Framework for API development.
 - Hosted on Robotic Marketer's cloud architecture.
- **Database:**
 - Uses MongoDB Atlas, version 8 or higher, at the M10+ tier with TLS.
 - Region of the database will be selected based on the location of the majority of users, to minimise latency.

2.5 Design and Implementation Constraints

The design and implementation of the system is subject to several constraints that guide the team's technology choices, in order to ensure compliance with legal and ethical standards, and sponsor requirements. These constraints determine the boundaries within which the project must be developed, helping maintain consistency and long-term sustainability. The following outlines the key limitations and requirements:

- **Accessibility Compliance:** The application must comply with WCAG 2.1 AA accessibility standards.
- **Technology Used:** Development must use Python 3.12+ and the Django 5.2+ REST Framework.
- **Data Privacy:** Must comply with GDPR and Australian Privacy Act. This can potentially limit what data can be stored.
- **API Usage:** Only publicly available datasets or APIs can be used. Proprietary or paid APIs are excluded unless the sponsor provides access.
- **LLM Usage:** No paid LLMs to be used for inference unless approved by the sponsor. Summarisation tasks must default to extraction/compactive methods using small local models.
- **Open-Source Licenses:** Any library or pretrained model used must be on an open-source license which allows for commercial use, such as the Apache 2.0 or MIT license.

2.6 User Documentation

Clear and accessible documentation will be delivered along with the software to support end users in effectively utilising the application and all its features. The documentation will include both written guides and embedded help features to ensure users have multiple ways to access the information. The following documentation will be used:

- **User Guide:** A concise guide available in PDF and HTML formats, covering:
 - How to search and filter for specific industries/locations
 - How to interpret the scores used
 - Known system limitations
- **Contextual Tooltips:** The application should contain inline tooltips for the following:
 - How the relevance score is calculated
 - What the engagement numbers mean
 - What semantic similarity is, and how it's calculated
- **FAQ:** The application should contain a FAQ section covering the most common errors, and how to address them.

2.7 Assumptions and Dependencies

The successful development of the system relies on several assumptions about resources, external services, and team member participation. These factors represent the conditions outside the direct control of the project team. Any changes to these assumptions may impact the scope or schedule of the project. The following are the key assumptions being made:

- **Availability of Public Resources:** The public APIs and datasets required for the system will remain available for the entire duration of the project.
- **Stability of Development Platforms:** Services used for the development such as GitHub, MongoDB Atlas, and Fly.io will remain stable and accessible.
- **Policy Stability:** No major changes to the API ToS, or data privacy regulations, such as the Australian Privacy Act, will occur during the development of the project.
- **External API Usage Limits:** APIs used, such as for Reddit and LinkedIn, will continue to support the needed query volumes within their free tier limits.
- **Team Availability:** Project team members will be consistently available to attend meetings and complete assigned tasks before the required deadlines.
- **Technical Resources:** Team devices will meet the baseline requirements required to run the needed NLP tasks, such as TF-IDF locally during development and testing.
- **Sponsor Feedback:** Sponsor feedback will be provided within 3 business days of a demonstration, to avoid blocking sprint progression.
- **Branding Assets:** Sponsor will supply the necessary branding assets, such as logos, to ensure the application's user interface is visually consistent with the rest of Robotic Marketer's platform.

3. System Features

3.1 System Feature 1 - Article retrieval

3.1.1 Description and Priority

- *This feature provides the ability to fetch articles and posts from multiple online platforms, including Reddit and LinkedIn. The retrieved content forms the foundation of the system, which enables further processing such as ranking and summarisation.*
- *Priority: High, article retrieval is the core functionality of the system and is essential for the minimum viable product (MVP).*

3.1.2 Stimulus/Response Sequences

- A user enters a keyword into the search bar.
- The system validates the input and queries connected external APIs.
- Retrieved results are consolidated and displayed to the user through the frontend.
- If one API fails, results from the remaining sources are still returned.
- If API rate limits are exceeded, the system notifies the user but continues to display partial results from available sources.

3.1.3 Functional Requirements

REQ-1.1: *The search bar must display a default placeholder "Keyword."*

REQ-1.2: *If one API fails, results from other APIs must still appear.*

REQ-1.3: *When API limits are exceeded, display a clear message identifying the missing*

source.

REQ-1.4: Empty or invalid queries must return an error message, not crash the system.

3.2 System Feature 2 - Relevance Scoring & Ranking

3.2.1 Description and Priority

- Hybrid scoring that combines lexical (TF-IDF), semantic (SBERT-MiniLM), and engagement (likes/comments) into a single normalized relevance score. **Priority: High.**

3.2.2 Stimulus/Response Sequences

- User submits a query → system computes per-document sub-scores → weighted fusion → returns ranked list with score breakdown per item.
- User adjusts filters (date range, domain, industry) → list re-ranks without a full refetch when possible (use cached vectors).

3.2.3 Functional Requirements

- **REQ-2.1:** The system shall compute TF-IDF cosine similarity between query and document text.
 - **REQ-2.2:** The system shall compute sentence-embedding cosine similarity using a small SBERT model (e.g., *MiniLM*).
 - **REQ-2.3:** The system shall ingest platform engagement signals (counts if available) and scale them to [0, 1] before fusion.
 - **REQ-2.4:** The UI shall display a hoverable breakdown of the score components (lex/sem/eng).
 - **REQ-2.5:** The system shall return ranked results within $\leq 1.5s$ median for a page of 20 items on staging data volume.
-

3.3 System Feature 3 - Filtering, Disambiguation & Facets - elaborate

3.3.1 Description and Priority

- Disambiguate ambiguous queries (e.g., “Apple”, “Jaguar”) via lightweight clarifying facets (industry, location, content type). **Priority: High.**

3.3.2 Stimulus/Response Sequences

- User searches ambiguous term → system suggests facet chips (e.g., “Technology”, “Automotive”) → user selects → results re-ranked.

3.3.3 Functional Requirements

- **REQ-3.1:** The system shall propose at least 2 facet suggestions when the query matches known ambiguous entities.
- **REQ-3.2:** The system shall allow filtering by date range, domain allow-list, industry, and language.
- **REQ-3.3:** The system shall persist the selected filters in the URL (query string) for shareable searches.

3.4 System Feature 4 - Summarization (Optional/Configurable)

3.4.1 Description and Priority: Provide short extractive summaries for quick scanning; optional and toggleable to control cost/performance. **Priority: Medium.**

3.4.2 Stimulus/Response Sequences

- *User opens result card → system displays a 2–3 sentence summary and highlighted matched phrases.*

3.4.3 Functional Requirements

- **REQ-4.1:** *The system shall generate a summary ≤ 400 characters using extractive methods by default.*
 - **REQ-4.2:** *Summaries must not include PII scraped from comments unless the source explicitly allows it; if present, redact.*
 - **REQ-4.3:** *Summarization can be disabled via an environment variable without a code change.*
-

3.5 System Feature 5 - Caching & Resilience

3.5.1 Description and Priority

- *Reduce latency and protect against rate limits by caching queries/results and degrading gracefully on upstream failure. **Priority: High.***

3.5.2 Stimulus/Response Sequences

- *Repeat query within cache TTL → serve cached results; stale-while-revalidate refresh runs asynchronously.*

3.5.3 Functional Requirements

- **REQ-5.1:** *The system shall cache normalized queries and their top-K results for at least 30 minutes (configurable).*
- **REQ-5.2:** *On third-party API outage, the system shall return partial results with a source status banner*
- **REQ-5.3:** *All outbound calls shall implement exponential backoff and jitter.*

4. External Interface Requirements

This section specifies all interfaces between the AI-powered web application and its environment, including user-facing UI, supported hardware, software dependencies, and communication/transport protocols. All requirements are written to be **clear, testable, and verifiable**.

4.1 User Interfaces

4.1.1 UI Principles & Compliance

- The system shall comply with **WCAG 2.1 AA** (keyboard operability, focus order, color contrast $\geq 4.5:1$, alt text, ARIA where applicable).
- The system shall implement **responsive design** for desktop ($\geq 1280\text{px}$), tablet ($\geq 768\text{px}$), and mobile ($\leq 480\text{px}$) breakpoints.

- The system shall follow sponsor brand guidelines for typography, color palette, and logo placement.

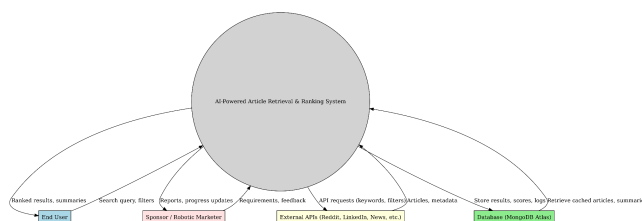
4.1.2 Core Screens

- **Home/Search**
 - The system shall display a keyword search input with placeholder text “Search topics, companies or keywords...”.
 - The system shall provide optional filters (date range, source, language, location/industry if available).
 - The system shall provide a “Search” action and show a loading indicator during retrieval.
- **Results Listing**
 - The system shall display ranked results with: title (clickable link), source, timestamp, snippet/summary, **lexical/semantic/engagement scores** (or a combined score), and badges for data quality (e.g., “Low engagement”).
 - The system shall provide sorting by **Relevance (default)**, **Newest**, and **Engagement**; and filtering by source.
 - The system shall support pagination or infinite scroll with deterministic ordering.
- **Detail View**
 - The system shall show full metadata, cached summary, and outbound link.
 - The system shall display an audit note when a source is unavailable/rate-limited.
- **Saved Items**
 - The system shall allow saving/unsaving results for later review (if in scope).
- **Global Elements**
 - The system shall provide a persistent header (Home, Saved, Settings, Help/Feedback).
 - The system shall present consistent, human-readable error messages (e.g., “LinkedIn data temporarily unavailable; showing results from other sources”).

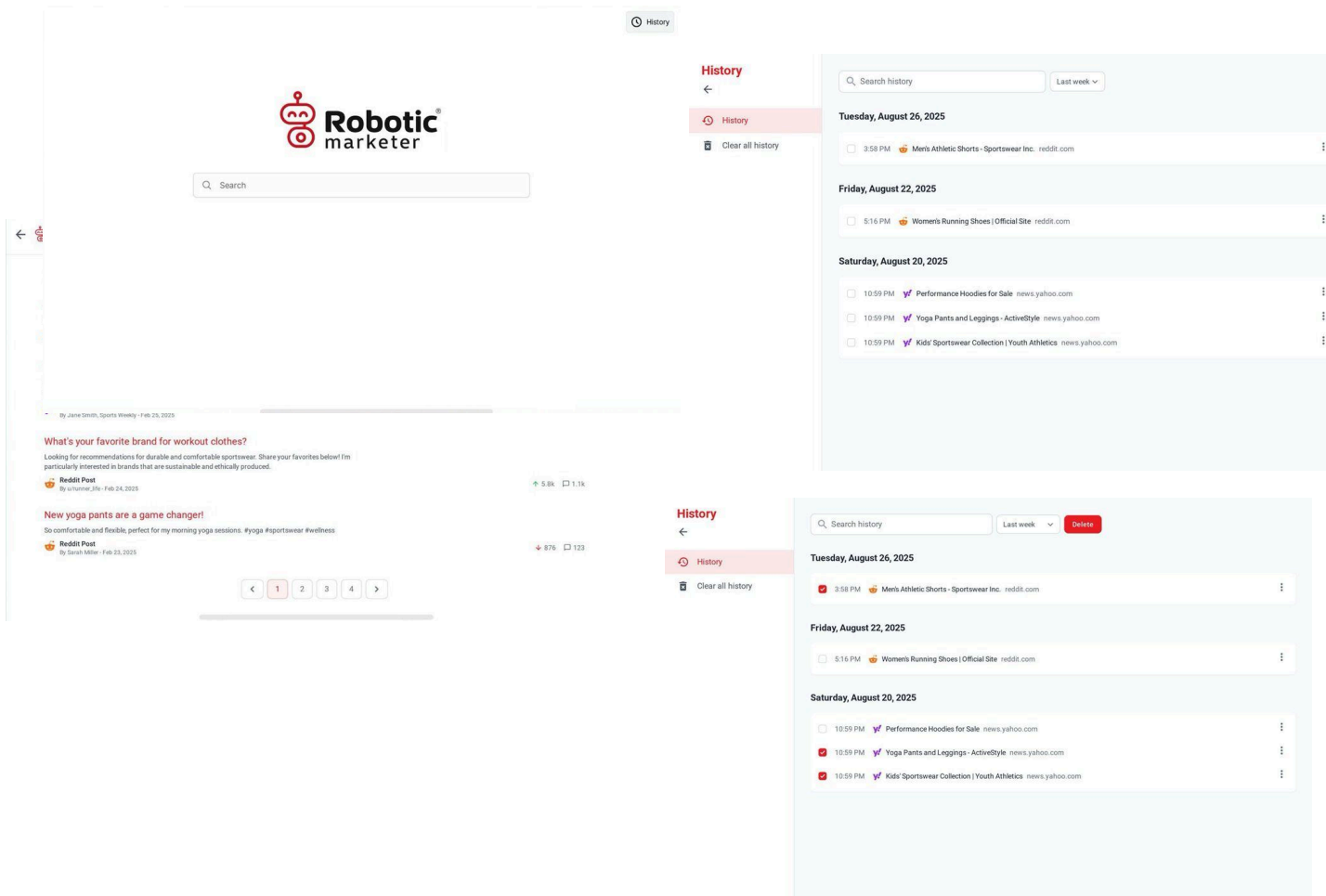
4.1.3 Interaction & Usability

- The system shall debounce keystrokes ($\geq 300\text{ms}$) to avoid excessive API calls for suggest/autocomplete (if implemented).
- The system shall persist user filters during navigation within a session.
- The system shall support i18n/l10n hooks for UI strings (future-ready).

4.1.4 Context Diagram



4.1.5 User Interface Design



4.2 Hardware Interfaces

Client Devices

- The system shall support modern browsers (latest two versions of Chrome, Edge, Safari, Firefox) on desktop and mobile devices with stable internet connectivity.

Server/Compute

- The system shall run on cloud infrastructure supporting autoscaling for API and NLP workloads.
- The system shall provide **≥10 GB** storage for model artifacts and cached results (expandable).
- (Optional) If a GPU is provisioned, the system should utilize it for transformer inference to reduce latency.

4.3 Software Interfaces

4.3.1 Internal Components

- **Frontend:** HTML5/CSS3/JavaScript (framework optional), responsive UI.
- **Backend:** Python 3.12+, **Django REST Framework** for RESTful APIs.
- **NLP/ML:**
 - TF-IDF vectorization (scikit-learn) and **SBERT MiniLM** for semantic similarity.

- Pluggable scoring pipeline (lexical, semantic, engagement, optional LLM score when permitted).
- **Database/Cache:**
 - MongoDB Atlas (articles, metadata, scores, telemetry).
 - Optional Redis for caching requests/results (if needed for performance).
- **Version Control/CI:** GitHub with protected branches and CI for lint/tests.

4.3.2 External APIs & Data Sources

- The system shall integrate with approved public APIs (e.g., Reddit, news APIs, others permitted by sponsor) and **shall not** use restricted/paid LLM APIs unless explicitly approved.
- The system shall store API credentials only in secure environment variables/secret manager (never client-side).
- The system shall honor each provider's **ToS, licensing, and rate limits**. If a provider changes terms or limits, the system shall degrade gracefully (Section 4.4).

4.3.3 Data Contracts

- The backend shall expose REST endpoints that return **JSON** with stable, versioned schemas, e.g.:

```
{
  "query": "renewable energy",
  "results": [{
    "id": "src:reddit:abc123",
    "title": "...",
    "url": "...",
    "source": "reddit",
    "published_at": "2025-08-10T12:05:00Z",
    "summary": "...",
    "scores": {"lexical": 0.62, "semantic": 0.79, "engagement": 0.43, "composite": 0.71}
  }],
  "meta": {"page": 1, "page_size": 20, "total": 124}
}
```

Backwards-incompatible API changes shall be introduced under a new versioned path (e.g., /api/v2/...).

4.4 Communications Interfaces

4.4.1 Transport & Formats

- All client–server communications shall use **HTTPS** with **TLS 1.2+**.
- The system shall use **REST** over HTTP(S) with **JSON** payloads (UTF-8).
- The system shall set appropriate cache headers for static assets and disable caching for sensitive API responses.

4.4.2 Authentication & Authorization

- External APIs: The system shall use API keys or OAuth2 as required by each provider.
- Internal APIs: If authentication is later required for users, the system shall support token-based auth (e.g., JWT) with expiration and refresh flows.

4.4.3 Reliability, Errors, and Rate-Limiting

- The system shall implement **timeouts** (default 10s) and **exponential backoff** retries (e.g., 2, 4, 8 seconds; max 3 attempts) for outbound API calls.

- On partial failures (e.g., one provider down), the system shall:
 - Return available results from other sources,
 - Include a machine-readable warning in response metadata,
 - Surface a human-readable notice in the UI.
- The system shall implement **circuit breaker** semantics to avoid hammering failing providers.
- The system shall log all outbound request status codes, durations, and failures for observability (PII-safe).

4.4.4 Performance Targets

- P50 search response $\leq 2.5s$, P95 $\leq 5s$ (assuming at least one provider available and cache warm).
- API throughput target: ≥ 20 **requests/sec** burst without degradation for MVP scope (tunable with infra).
- Maximum payload for a single page of results: ≤ 512 **KB** compressed.

4.4.5 Security & Privacy

- The system shall never expose secrets in client code or responses.
- The system shall sanitize all inbound parameters to prevent injection, and escape content from external sources to prevent XSS.
- The system shall process and store only **publicly available** data unless otherwise approved, and shall comply with **Australian Privacy Act 1988** and applicable regulations (GDPR/CCPA where relevant to use).
- The system shall support data retention policies (e.g., auto-purge cached content after N days configurable).

4.4.6 Observability

- The system shall provide structured logs (JSON) with trace IDs for request correlation.
- The system shall expose basic health endpoints (e.g., /healthz, /readiness) for deployment orchestration.
- The system shall capture metrics (request count/latency, error rates, cache hit ratio) for performance tuning.

4.4.7 Traceability Considerations

To ensure consistency from specification through verification, all external interface requirements are traceable to functional and non-functional specifications and to corresponding test artefacts. Specifically, user-interface requirements (e.g., search, ranking, and summarisation views) are mapped to the relevant System Features to prevent orphaned UI behaviour and to guarantee end-to-end coverage. Software and communications interface definitions (e.g., external APIs, protocols, payload schemas) are cross-referenced to the Non-Functional Requirements for performance, security, privacy, and availability to confirm that interface contracts align with system quality attributes and regulatory constraints.

Each requirement expressed as a “shall” statement is written to be objectively verifiable and is linked to one or more test categories, UI tests (functional and accessibility), API contract tests (schema/semantic validation against providers), and integration tests with stubbed/mock external services, to enable measurable acceptance criteria and regression safety over time.

4.4.1 Performance Requirements

When a user hits the search button the app should try to show the first useful ranked results in about 4 seconds under normal conditions. To keep pages results are split into pages, up to 10 articles, instead of sending one huge pile which helps avoid long waits and big downloads. While data is being fetched the UI must show a clear loading icon so the user can see work is happening and it should drop partial results as soon as they appear rather than waiting for every source to reply. If the same query is asked again within a short span the system may reuse the last answer to speed things up while it quietly brings fresher data in the background. Going back to recent results should feel instant. There is no needless re-fetch required. Slow or unreachable sources must not hold up the page other sources keep rendering. The interface ought to stay usable on typical laptop and mobile connections and stay responsive to scrolling opening details paging while extra data loads.

4.5 Safety Requirements

The program only reads stuff that everybody can see online. It never writes, edits or erases anything on other sites, and it won't try to slip past paywalls or get into locked pages. Every call follows the host's rules and the law. Keys and passwords stay on the server so the user's browser never sees them. If one source goes down, gets limited, or throws an error, the app should still show a simple message, let the user try again, and keep any results that did come back from other places. All inputs and outputs get cleaned up so no one can inject bad code. The traffic runs through HTTPS and logs purposefully leaving out personal clues, no usernames, emails or other IDs. Any saved search history ought to be tiny, explained clearly, and easy to delete. It also respects user privacy during interaction. Finally, the tool should ask for data gently, using basic rate limit or back off logic so it doesn't swamp the outside services.

4.6 Security Requirements

To ensure the protection of user data and the integrity of the system, the application must follow strong security practices. These measures address potential threats such as unauthorized access, data leaks, and malicious attacks, while also ensuring compliance with privacy regulations. The following security requirements define the standards that must be implemented throughout the system:

- **HTTPS Enforcement:** All communication between clients and the server must be encrypted using HTTPS to ensure confidentiality and integrity of user data.
- **Input/Output Sanitization:** All user inputs and outputs must be sanitized to prevent injection attacks (e.g., XSS).
- **API Key Protection:** API keys and other sensitive credentials must be stored server-side and never exposed in client code. Access to external APIs must be secured.
- **User Authentication:** Users must authenticate with a secure system (e.g., Django authentication framework) before accessing any personalized features. Passwords must be hashed and stored securely using industry-standard methods.
- **Privacy Compliance:** The application must adhere to relevant privacy regulations, such as GDPR or local data protection laws, especially when handling user data.

Security Certifications: Follow best practices for web application security, including OWASP Top 10 mitigation strategies. Optional security audits or penetration testing may be conducted before deployment.

4.7 Software Quality Attributes

To ensure the application meets user expectations and functions reliably across different environments, several software quality attributes must be addressed. These attributes define the standards for usability, performance, maintainability, and overall system behavior, providing a foundation for a robust and user-friendly application. The following quality attributes outline the key requirements for the system:

Usability:

- *Clear headings and labels for all UI components.*
- *Visible empty states and error messages to guide users.*
- *Consistent layout across all pages and devices.*

Compatibility/Portability:

- *Application must run seamlessly on major browsers: Chrome, Edge, and Safari.*
- *Must be accessible on laptops, desktops, and mobile devices.*

Reliability:

- *Handle API errors and timeouts gracefully.*
- *UI should never crash; provide retry buttons or fallback messages when data retrieval fails.*

Maintainability:

- *Use semantic HTML and modular CSS for easy updates.*
- *Code should be structured in short, well-defined components for easier modification.*

Performance:

- *The accuracy that its related to the search words must above 50%*
- *The Home-page when the website start must load within 5 seconds not more*
- *The search result must take no longer than 3-5 seconds*

Testability:

- *Unit tests must cover backend AI scoring logic and API interactions.*
- *Integration tests must validate end-to-end data flow from user input to displayed results.*

5. Other Requirements

In addition to functional and quality requirements, the system must satisfy several other considerations that support long-term usability, compliance, and flexibility. These requirements cover aspects such as database design, internationalization, legal obligations, and code reuse, ensuring the application is robust, adaptable, and maintainable over time.

Database Requirements:

- Use a non-relational database
- Ensure indexing for faster query retrieval.
- Showing max 10 articles

Internationalization:

- Support English initially; design a structure for future multi-language support.

Legal Requirements:

- Ensure only publicly available data is collected. Avoid violating third-party API terms of service.
- Any agreements signed with partner organizations (IP, NDA) must be adhered to.

Reuse Objectives:

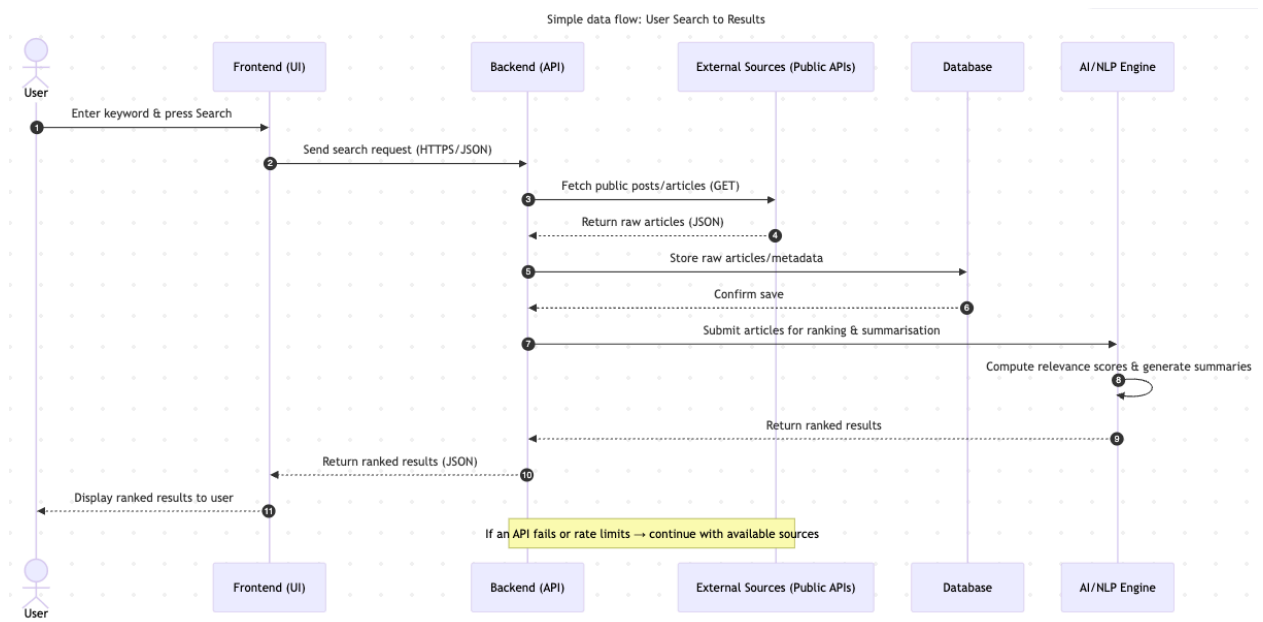
- Codebase should be modular to allow reuse in future AI-powered web applications or data mining tools.

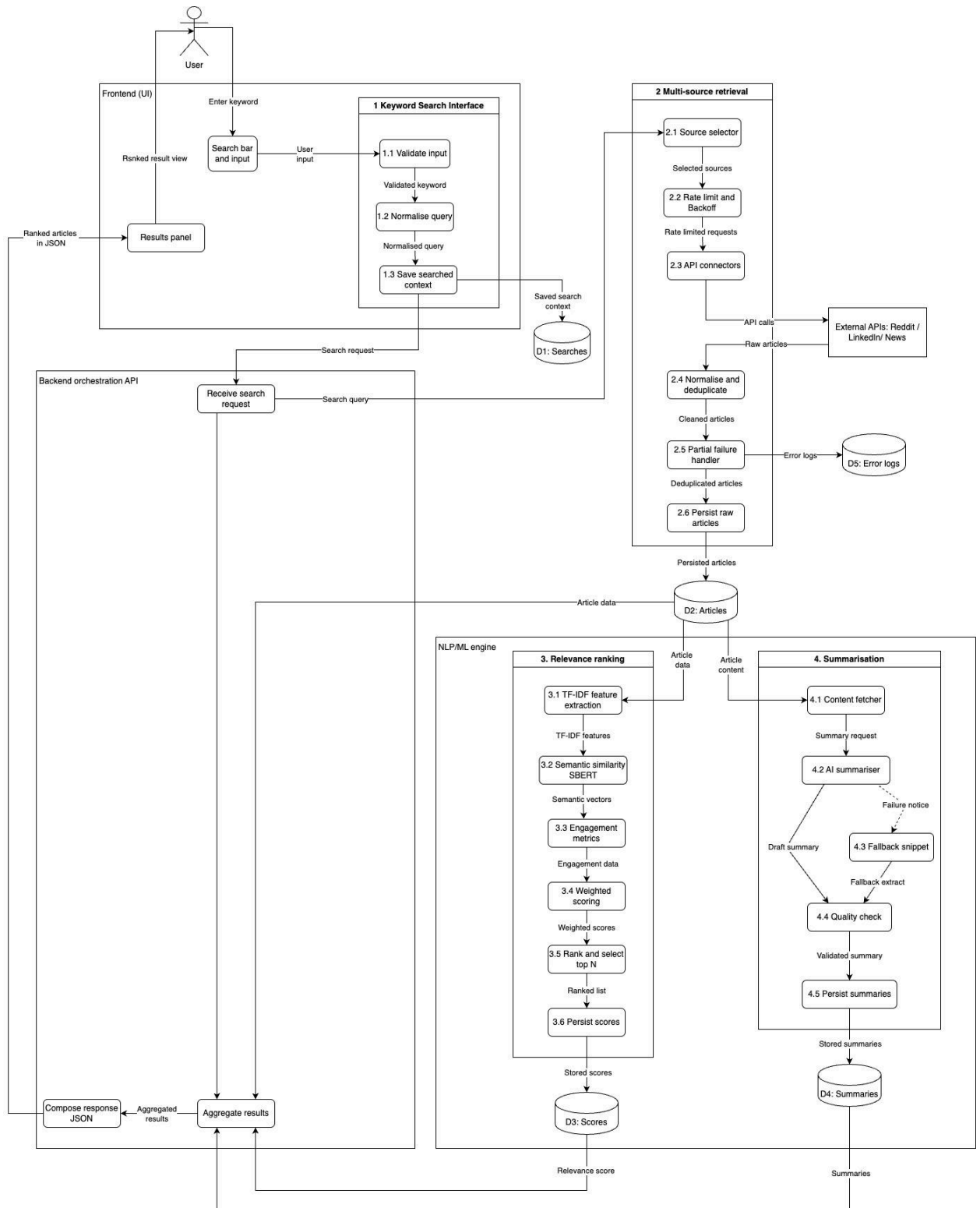
Appendix A: Glossary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

- Lexical Score: Similarity based on word frequency features (e.g., TF-IDF cosine)
- Semantic Score: Similarity based on sentence embeddings (e.g., SBERT MiniLM).
- Engagement Score: Normalized metric from platform interactions (likes, comments) when available.
- Facet: A clarifying filter (industry, location, date) used to disambiguate queries.

Appendix B: Analysis Models





Appendix C: Issues List

Client/Sponsor feedback

Meeting date and time

- Date: Every Thursday

- *Time: 5pm, via Microsoft Teams*
- *Sponsor: Robotic Marketer*
- *Team: Group 22*

Document shared with sponsors:

- *1st of September 2025*
- *4th of September 2025*

Feedback received *(most recent meeting)*

- *Meeting date/time: Thursday, 04/09/2025, 5pm, via Microsoft Teams*
- *Attendees: Anirudh, Khawaja, Eric, Yousef, Amr, Edward, Miroslav (Sponsor), Humayun (Sponsor)*

Summary of feedback:

- *User interface design should include location and industry fields on the search Homepage.*
- *For every heading, begin with a brief introduction/description before listing details.*
- *Provide ideas in paragraph format, with bullet points only used as a supplementary.*
- *Elaborate further on the product features section to add depth and clarity.*
- *Maximum data to display is 10 articles per search.*
- *Must get the license if not for commercial use for any external resources like hugging face, etc*
- *The loading page must load a maximum of 5 minutes, loading page no more than 3-5 minutes, and accuracy above 50%.*
- *Place the explanation of diagrams after the diagrams rather than before.*

Team response/action plan

- ***UI adjustments:*** *We will update the homepage design to include location and industry fields in the main search interface for the next iteration.*
- ***Section structure:*** *All headings will begin with a short introductory paragraph in the latest documents and future drafts to improve readability and alignment with expectations.*
- ***Formatting approach:*** *We will shift the majority of ideas into paragraph form, reserving bullet points only for supporting or supplementary details.*
- ***Product features:*** *The product features section will be expanded with more detail, covering functionality, use cases, and benefits to add depth and clarity.*
- ***Article limit:*** *The system will be configured to display a maximum of 10 articles per search to keep results concise and relevant.*

- **Licensing compliance:** *We will confirm licensing requirements for external tools (e.g., Hugging Face models) and document their usage appropriately to ensure compliance.*
- **Performance criteria:** *We will set system benchmarks to ensure results load within 3–5 minutes, with accuracy maintained above 50%. These will be tracked as part of testing and quality assurance.*
- **Diagram placement:** *Diagrams will be presented first, followed by explanatory text, to make the document flow more intuitive and visually clear.*